# Davidson_Dataset_Plot

April 26, 2026

Problem Number 1:-

(a) Write a program to draw a bar graph using pandas and matplotlib in the given dataset.
(b) Merge classes 0 and 1, and rename class 2 as 0

Name - Aditya Kumar Garg

University Roll No- 2023827

[1]:
```
import matplotlib.pyplot as plt
import pandas as pd
```

[2]:
```
df = pd.read_csv('labeled_data.csv')
df.describe()
df.head()
```

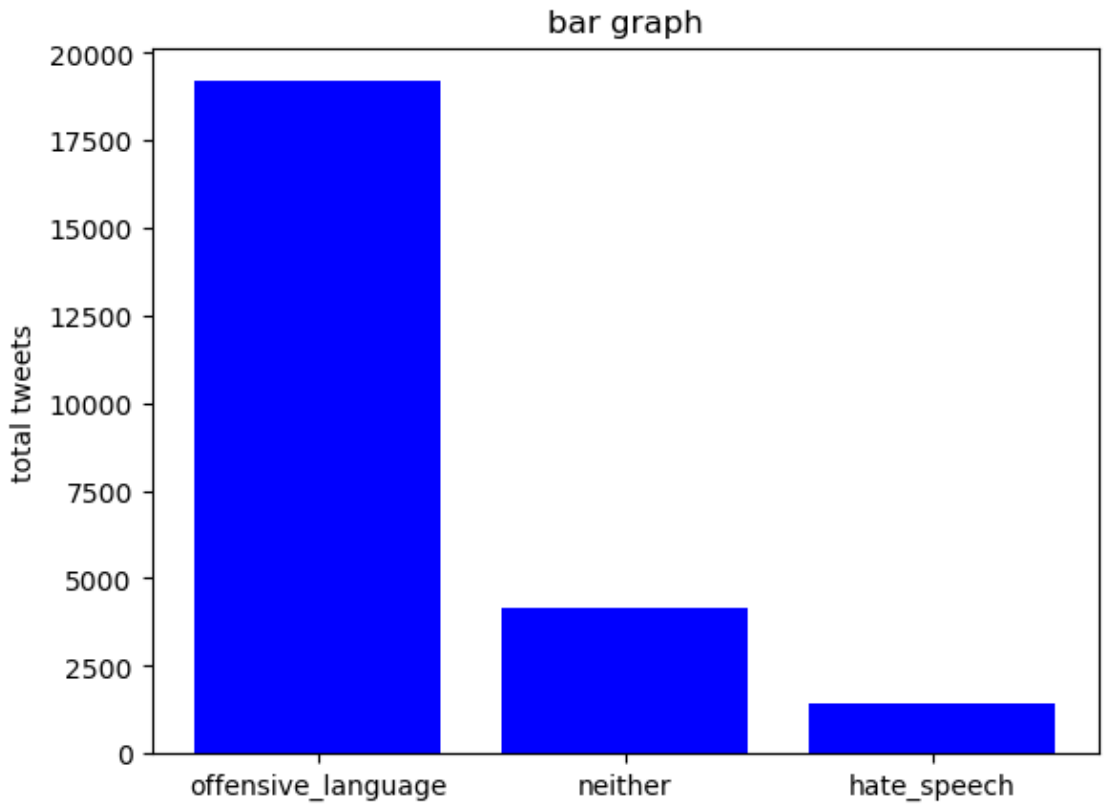
[2]:
```
   Unnamed: 0  count  hate_speech  offensive_language  neither  class  \
0           0      3            0                   0        3      2
1           1      3            0                   3        0      1
2           2      3            0                   3        0      1
3           3      3            0                   2        1      1
4           4      6            0                   6        0      1

                                                tweet
0  !!! RT @mayasolovely: As a woman you shouldn't…
1  !!!!! RT @mleew17: boy dats cold…tyga dwn ba…
2  !!!!!!! RT @UrKindOfBrand Dawg!!!! RT @80sbaby…
3  !!!!!!!!! RT @C_G_Anderson: @viva_based she lo…
4  !!!!!!!!!!!!! RT @ShenikaRoberts: The shit you…
```

[3]:
```
class_counts=df['class'].value_counts()
print(class_counts)
```

```
class
1    19190
2     4163
0     1430
Name: count, dtype: int64
```

[4]:
```
tweets = ["offensive_language","neither","hate_speech"]
```

[5]:
```
plt.bar(tweets,class_counts,color='blue')
plt.ylabel('total tweets')
plt.title('bar graph')
plt.show()
```

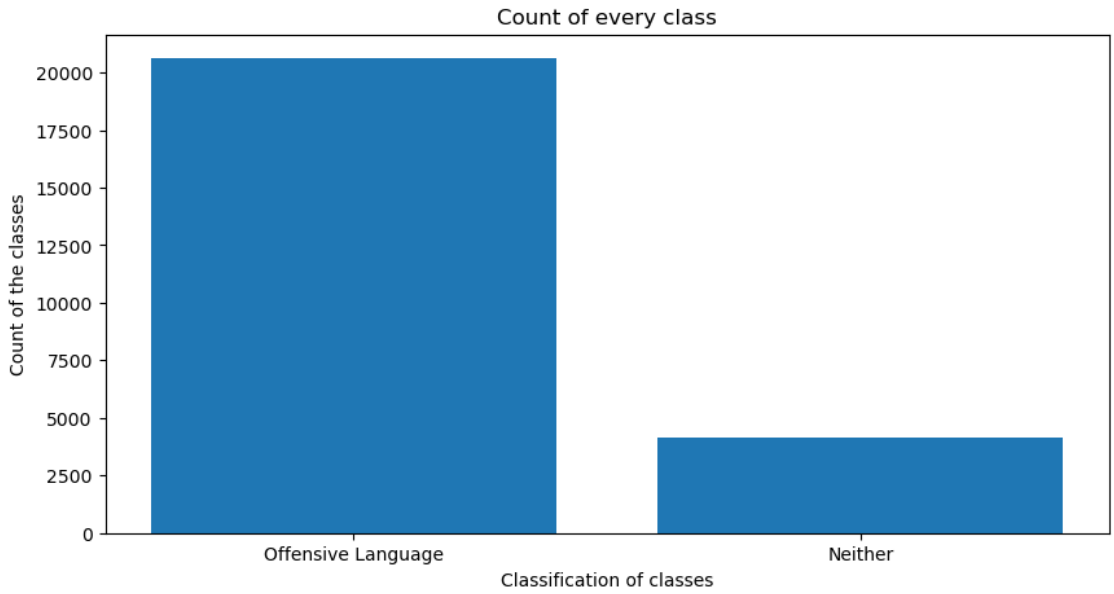



[6]:
```
df['class'] = df['class'].replace({0:1,1:1,2:0})
```

[7]:
```
class_counts_new = df['class'].value_counts()
print(class_counts_new)
```

```
class
1    20620
0     4163
Name: count, dtype: int64
```

[8]:
```
plt.figure(figsize=(10,5))
plt.title("Count of every class")
plt.xlabel("Classification of classes")
plt.ylabel("Count of the classes")
plt.bar(['Offensive Language','Neither'], class_counts_new)
plt.show()
```


Count of every class
Count of the classes
20000
17500
15000
12500
10000
7500
5000
2500
0
Offensive Language
Neither
Classification of classes

# implement_similarity_functions

April 26, 2026

Problem Number 2:-

To implement various similarity functions for attribute tuples stored as .csv file – Euclidean, Manhattan, Cosine, Minikowski.

Name - Aditya Kumar Garg

University Roll No- 2023827

[1]:
```
import pandas as pd
```

[2]:
```
input_data= pd.read_csv('similarity.csv')
```

[3]:
```
input_data
```

[3]:

| | INPUT1 | INPUT2 | INPUT3 | INPUT4 | INPUT5 |
|---|---|---|---|---|---|
| 0 | 2 | 6 | 8 | 4 | 9 |
| 1 | 1 | 4 | 6 | 6 | 7 |
| 2 | 6 | 9 | 12 | 34 | 6 |
| 3 | 5 | 6 | 3 | 6 | 1 |
| 4 | 2 | 4 | 6 | 7 | 10 |

[4]:
```
arr=input_data.values
```

[5]:
```
arr
```

[5]:
```
array([[ 2,  6,  8,  4,  9],
       [ 1,  4,  6,  6,  7],
       [ 6,  9, 12, 34,  6],
       [ 5,  6,  3,  6,  1],
       [ 2,  4,  6,  7, 10]])
```

[6]:
```
len(arr)
```

[6]:
```
5
```

EUCLIDEAN DISTANCE

[7]:
```
import numpy as np
```

[8]:
```
def euclidean_distance_function(x,y):
    return np.sqrt(np.sum((x-y)**2))
```

[9]:
```
for i in range(len(arr)):
    for j in range(i+1,len(arr)):
        print("euclidean distance:",euclidean_distance_function(arr[i],arr[j]))
```

```
euclidean distance: 4.123105625617661
euclidean distance: 30.822070014844883
euclidean distance: 10.099504938362077
euclidean distance: 4.242640687119285
euclidean distance: 29.512709126747414
euclidean distance: 8.06225774829855
euclidean distance: 3.3166247903554
euclidean distance: 30.0
euclidean distance: 28.6705423736629
euclidean distance: 10.198039027185569
```

MANHATTAN DISTANCE

[10]:
```
def manhattan_distance_function(x,y):
    return np.sum(np.abs(x-y))
```

[11]:
```
for i in range(len(arr)):
    for j in range(i+1,len(arr)):
        print("manhattan distance:",manhattan_distance_function(arr[i],arr[j]))
```

```
manhattan distance: 9
manhattan distance: 44
manhattan distance: 18
manhattan distance: 8
manhattan distance: 45
manhattan distance: 15
manhattan distance: 5
manhattan distance: 46
manhattan distance: 46
manhattan distance: 18
```

COSINE DISTANCE

[12]:
```
def cosine_distance_function(x,y):
    return np.dot(x,y)/(np.linalg.norm(x)*np.linalg.norm(y))
```

[13]:
```
for i in range(len(arr)):
    for j in range(i+1,len(arr)):
        print("cosine distance:",cosine_distance_function(arr[i],arr[j]))
```

```
cosine distance: 0.9666923997261102
cosine distance: 0.6513461157653094
cosine distance: 0.7023398445660594
```

```
cosine distance: 0.9557114093815584
cosine distance: 0.8039520564087513
cosine distance: 0.7406466748406141
cosine distance: 0.9869419619190055
cosine distance: 0.836930208775195
cosine distance: 0.7658902556978057
cosine distance: 0.7022059740544226
```

[14]:
```
from sklearn.metrics.pairwise import cosine_similarity
```

[15]:
```
cosine_similarity(input_data)
```

[15]:
```
array([[1.        , 0.9666924 , 0.65134612, 0.70233984, 0.95571141],
       [0.9666924 , 1.        , 0.80395206, 0.74064667, 0.98694196],
       [0.65134612, 0.80395206, 1.        , 0.83693021, 0.76589026],
       [0.70233984, 0.74064667, 0.83693021, 1.        , 0.70220597],
       [0.95571141, 0.98694196, 0.76589026, 0.70220597, 1.        ]])
```

MINKOWSKI DISTANCE

[16]:
```
def minkowski_distance_function(x,y,p):
    return (np.sum(np.abs(x-y)**p))**(1/p)
```

[17]:
```
for i in range(len(arr)):
    for j in range(i+1,len(arr)):
        print("minkowski distance:
 ↪",minkowski_distance_function(arr[i],arr[j],1))
```

```
minkowski distance: 9.0
minkowski distance: 44.0
minkowski distance: 18.0
minkowski distance: 8.0
minkowski distance: 45.0
minkowski distance: 15.0
minkowski distance: 5.0
minkowski distance: 46.0
minkowski distance: 46.0
minkowski distance: 18.0
```

[18]:
```
for i in range(len(arr)):
    for j in range(i+1,len(arr)):
        print("minkowski distance:
 ↪",minkowski_distance_function(arr[i],arr[j],2))
```

```
minkowski distance: 4.123105625617661
minkowski distance: 30.822070014844883
minkowski distance: 10.099504938362077
minkowski distance: 4.242640687119285
minkowski distance: 29.512709126747414
```

```
minkowski distance: 8.06225774829855
minkowski distance: 3.3166247903554
minkowski distance: 30.0
minkowski distance: 28.6705423736629
minkowski distance: 10.198039027185569
```

# pandas_for_preprocessing

April 26, 2026

Problem Number 3:-

Use Pandas for preprocessing, Matplotlib/Seaborn for visualization.

Name - Aditya Kumar Garg

University Roll No- 2023827

[1]:
```
import pandas as pd
```

[2]:
```
df = pd.read_csv("sales_data.csv")
print(df.head())
```

```
         Date Region Product   Sales  Profit
0  2023-01-05  North  Laptop  1200.0   200.0
1  2023-01-10  South   Phone   800.0   150.0
2  2023-02-12    NaN  Tablet     NaN   120.0
3  2023-02-18   West  Laptop  1500.0     NaN
4  2023-03-03  North   Phone   700.0   130.0
```

[3]:
```
df.isnull().sum()
df.fillna(df.mean(numeric_only=True), inplace=True)
print(df)
```

```
          Date Region Product   Sales  Profit
0   2023-01-05  North  Laptop  1200.0   200.0
1   2023-01-10  South   Phone   800.0   150.0
2   2023-02-12    NaN  Tablet  1010.0   120.0
3   2023-02-18   West  Laptop  1500.0   186.0
4   2023-03-03  North   Phone   700.0   130.0
5   2023-03-15  South     NaN   650.0   140.0
6   2023-04-07   East  Laptop  1700.0   350.0
7   2023-04-20   West   Phone   900.0   180.0
8   2023-05-02  North  Tablet   500.0   100.0
9   2023-05-18    NaN  Laptop  1600.0   320.0
10  2023-06-10   East   Phone  1010.0   170.0
11  2023-06-25   West  Tablet   550.0   186.0
```

[4]:
```
df = df.dropna()
print(df)
```

```
          Date Region Product   Sales  Profit
0   2023-01-05  North  Laptop  1200.0   200.0
1   2023-01-10  South   Phone   800.0   150.0
3   2023-02-18   West  Laptop  1500.0   186.0
4   2023-03-03  North   Phone   700.0   130.0
6   2023-04-07   East  Laptop  1700.0   350.0
7   2023-04-20   West   Phone   900.0   180.0
8   2023-05-02  North  Tablet   500.0   100.0
10  2023-06-10   East   Phone  1010.0   170.0
11  2023-06-25   West  Tablet   550.0   186.0
```

[5]:
```python
df['Date'] = pd.to_datetime(df['Date'])
print(df)
```

```
         Date Region Product   Sales  Profit
0  2023-01-05  North  Laptop  1200.0   200.0
1  2023-01-10  South   Phone   800.0   150.0
3  2023-02-18   West  Laptop  1500.0   186.0
4  2023-03-03  North   Phone   700.0   130.0
6  2023-04-07   East  Laptop  1700.0   350.0
7  2023-04-20   West   Phone   900.0   180.0
8  2023-05-02  North  Tablet   500.0   100.0
10 2023-06-10   East   Phone  1010.0   170.0
11 2023-06-25   West  Tablet   550.0   186.0
```

[6]:
```python
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
print(df)
```

```
         Date Region Product   Sales  Profit  Year  Month
0  2023-01-05  North  Laptop  1200.0   200.0  2023      1
1  2023-01-10  South   Phone   800.0   150.0  2023      1
3  2023-02-18   West  Laptop  1500.0   186.0  2023      2
4  2023-03-03  North   Phone   700.0   130.0  2023      3
6  2023-04-07   East  Laptop  1700.0   350.0  2023      4
7  2023-04-20   West   Phone   900.0   180.0  2023      4
8  2023-05-02  North  Tablet   500.0   100.0  2023      5
10 2023-06-10   East   Phone  1010.0   170.0  2023      6
11 2023-06-25   West  Tablet   550.0   186.0  2023      6
```

[7]:
```python
df = df[df['Sales'] > 500]
print(df)
```

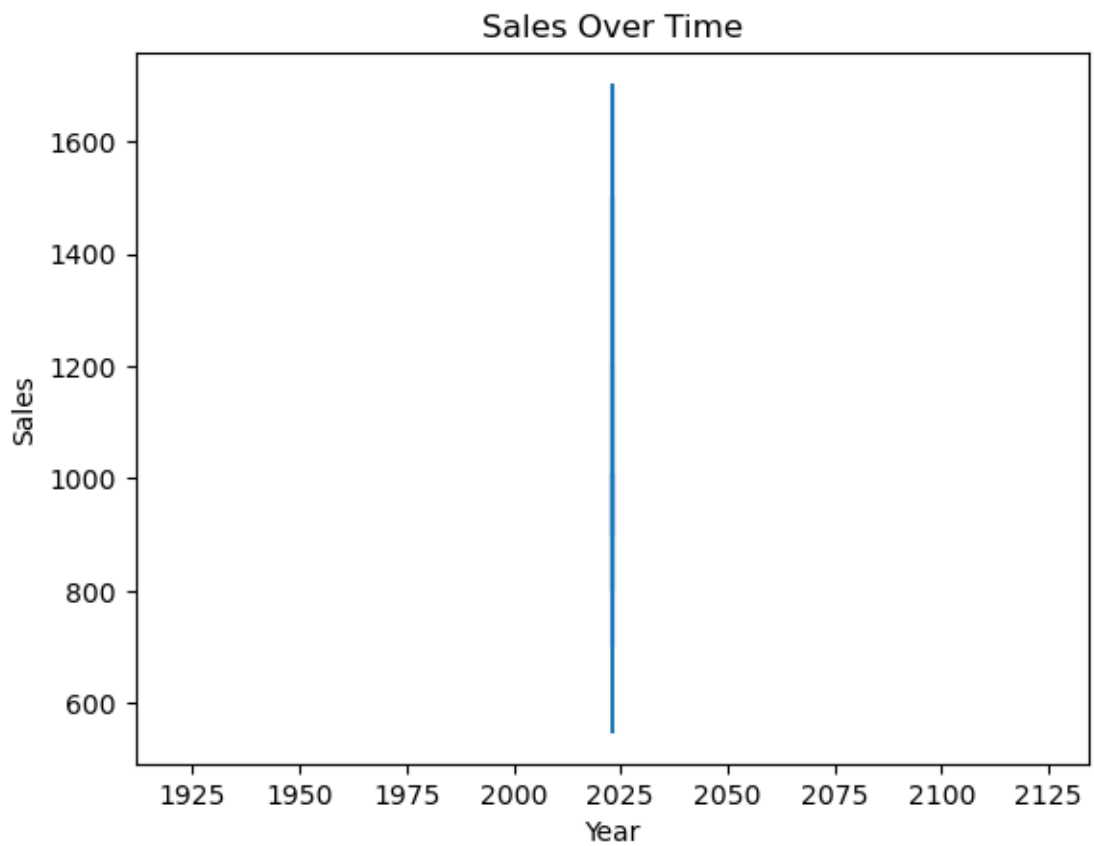
```
         Date Region Product   Sales  Profit  Year  Month
0  2023-01-05  North  Laptop  1200.0   200.0  2023      1
1  2023-01-10  South   Phone   800.0   150.0  2023      1
3  2023-02-18   West  Laptop  1500.0   186.0  2023      2
4  2023-03-03  North   Phone   700.0   130.0  2023      3
6  2023-04-07   East  Laptop  1700.0   350.0  2023      4
```

```
7  2023-04-20   West   Phone   900.0   180.0  2023      4
10 2023-06-10   East   Phone  1010.0   170.0  2023      6
11 2023-06-25   West  Tablet   550.0   186.0  2023      6
```

[8]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

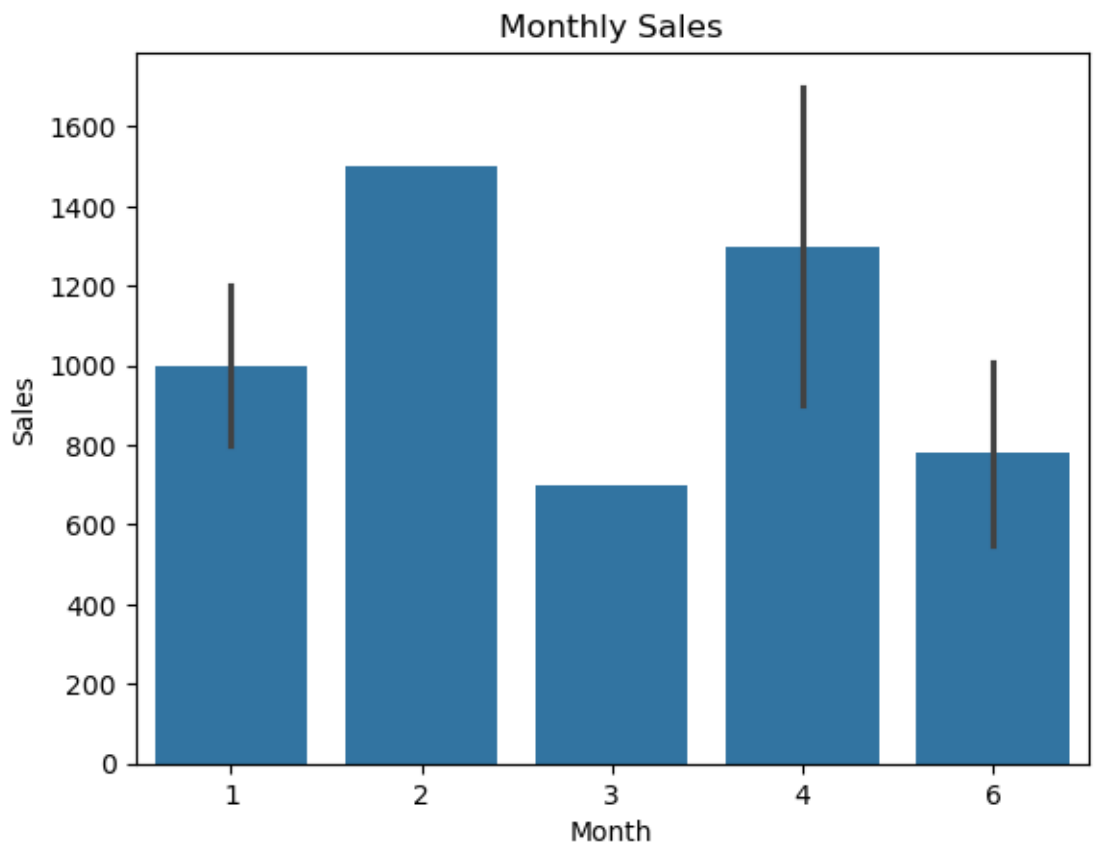
Line Plot

[9]:
```
plt.plot(df['Year'], df['Sales'])
plt.xlabel("Year")
plt.ylabel("Sales")
plt.title("Sales Over Time")
plt.show()
```



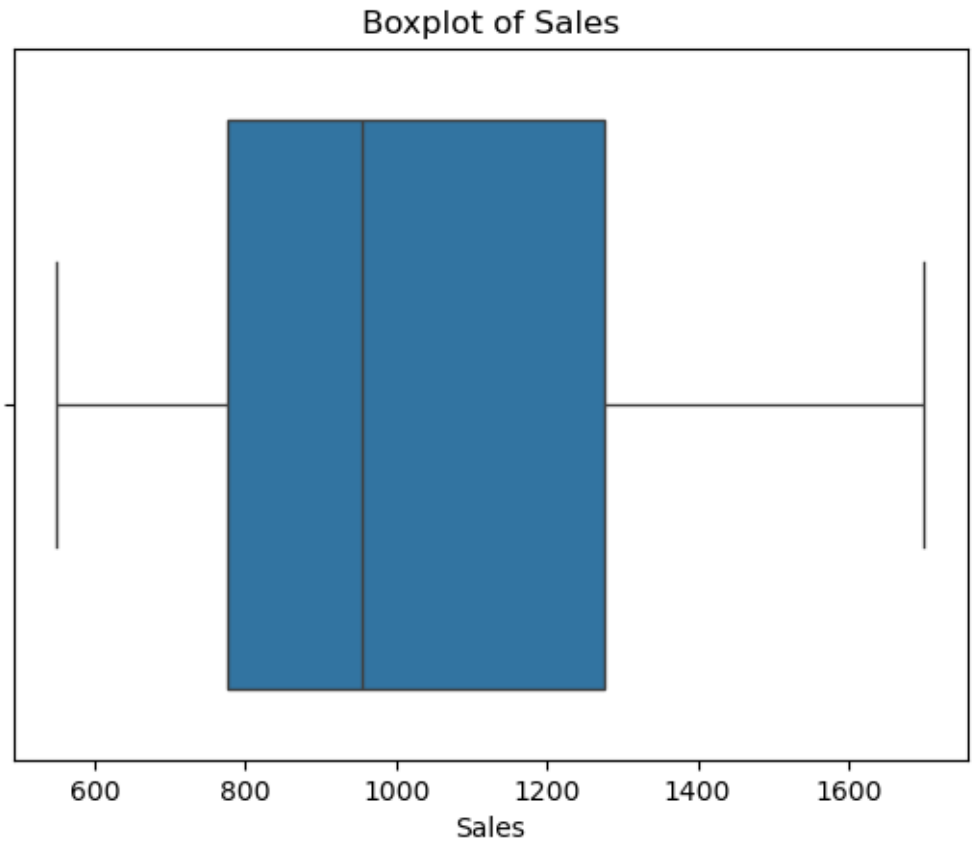

Bar Plot

[10]:
```
sns.barplot(x='Month', y='Sales', data=df)
plt.title("Monthly Sales")
plt.show()
```



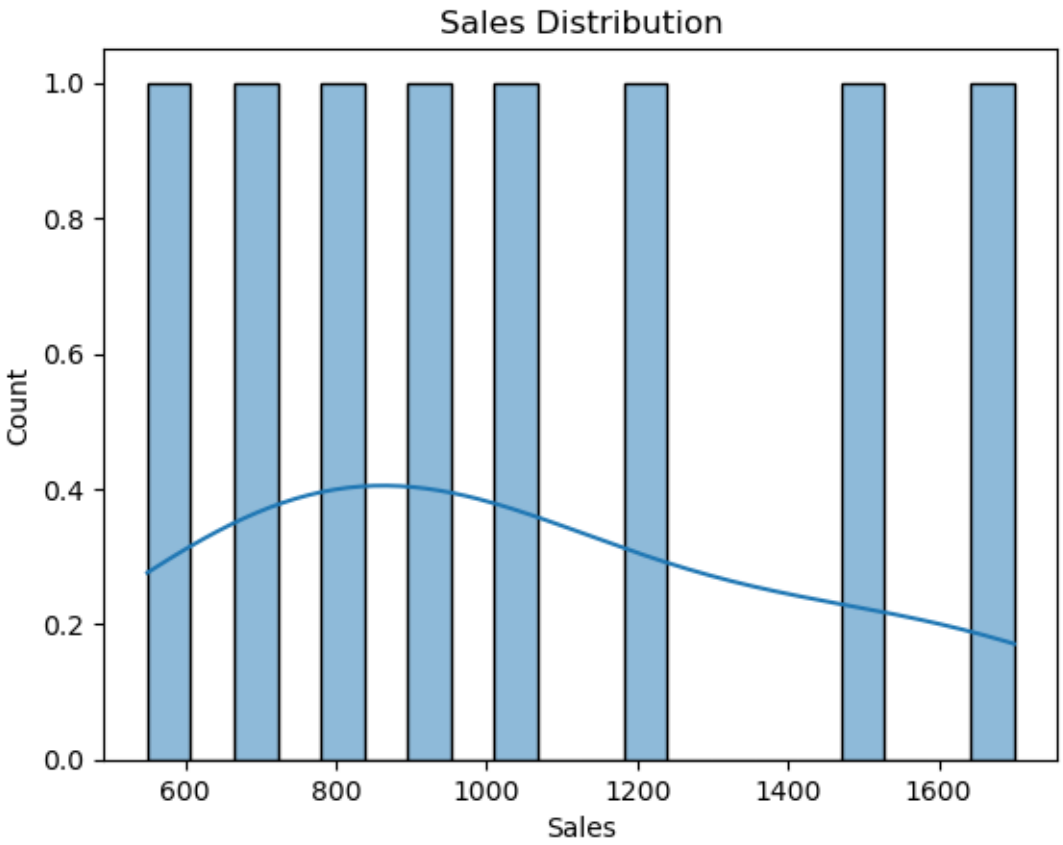

Box Plot

[11]:
```
sns.boxplot(x=df['Sales'])
plt.title("Boxplot of Sales")
plt.show()
```




Histogram

[12]:
```
sns.histplot(df['Sales'], bins=20, kde=True)
plt.title("Sales Distribution")
plt.show()
```


Sales Distribution
1.0
0.8
0.6
0.4
0.2
0.0
Count
600
800
1000
1200
1400
1600
Sales

# linear_regression_sklearn

April 26, 2026

Problem Number 4:-

Implement a linear regression model using Scikit-learn and evaluate Mean Squared Error (MSE).

Name - Aditya Kumar Garg

University Roll No- 2023827

[1]:
```
import pandas as pd
```

[2]:
```
#load the data
data = pd.read_csv('Housing.csv')
```

[3]:
```
data
```

[3]:
```
        price  area  bedrooms  bathrooms  stories mainroad guestroom basement  \
0    13300000  7420         4          2        3      yes        no       no
1    12250000  8960         4          4        4      yes        no       no
2    12250000  9960         3          2        2      yes        no      yes
3    12215000  7500         4          2        2      yes        no      yes
4    11410000  7420         4          1        2      yes       yes      yes
..        …     …       …         …       …      …        …       …
540   1820000  3000         2          1        1      yes        no      yes
541   1767150  2400         3          1        1       no        no       no
542   1750000  3620         2          1        1      yes        no       no
543   1750000  2910         3          1        1       no        no       no
544   1750000  3850         3          1        2      yes        no       no

    hotwaterheating airconditioning  parking prefarea furnishingstatus
0                no             yes        2      yes        furnished
1                no             yes        3       no        furnished
2                no              no        2      yes   semi-furnished
3                no             yes        3      yes        furnished
4                no             yes        2       no        furnished
..              …              …       …       …               …
540              no              no        2       no      unfurnished
541              no              no        0       no   semi-furnished
542              no              no        0       no      unfurnished
543              no              no        0       no        furnished
544              no              no        0       no      unfurnished
```

```
[545 rows x 13 columns]
```

[4]:
```
#Extract the input and output
x= data['area']
y= data['price']
```

[5]:
```
# 2d-array
import numpy as np
x = np.array(x).reshape(-1,1)
y = np.array(y).reshape(-1,1)
```

[6]:
```
print(f"Total inputs - {len(x)}")
print(f"Total outputs - {len(y)}")
```

```
Total inputs - 545
Total outputs - 545
```

[7]:
```
#split the dataset using train_test_split() method
from sklearn.model_selection import train_test_split
```

[8]:
```
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.2)
```

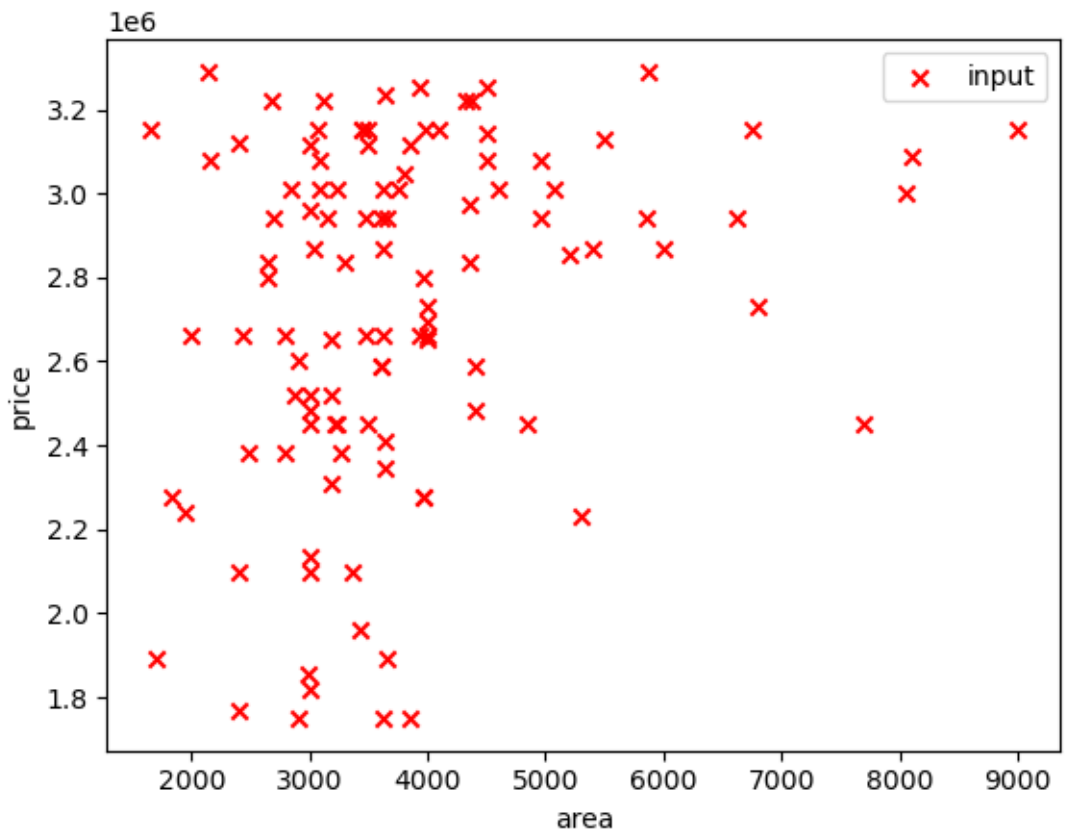
[9]:
```
print(f"Total Training inputs - {len(x_train)}")
print(f"Total Testing outputs - {len(x_test)}")
```

```
Total Training inputs - 436
Total Testing outputs - 109
```

[10]:
```
# Manually spiliting the dataset
# slicing
x_test= x[:436]
x_train= x[436:]
y_test= y[:436]
y_train= y[436:]
```

[11]:
```
import matplotlib.pyplot as plt
```

[12]:
```
plt.scatter(x_train,y_train,c='red',marker='x',label="input")#label used to␣
↪tell 'x' sybolises input
plt.xlabel("area")
plt.ylabel("price")
plt.legend()#to show label
plt.show()
```




[13]:
```
# make linear regression using sklearn
from sklearn.linear_model import LinearRegression
```

[14]:
```
# object to LinearRegression
model_linreg = LinearRegression()
```

[15]:
```
# fit the model
model_linreg.fit(x_train, y_train)
```

[15]:
```
LinearRegression()
```

[16]:
```
# prediction of the model on a single input value
y_pred_val = model_linreg.predict([[4067]])
y_pred_val
```

[16]:
```
array([[2744567.53746759]])
```

[17]:
```
y_pred = model_linreg.predict(x_test)
```

[18]:
```
y_pred
```

[18]:
```
array([[3025727.88427433],
       [3154862.07278681],
       [3238715.44195077],
       [3032436.15380744],
       [3025727.88427433],
       [3032436.15380744],
       [3122997.79250451],
       [3761960.46553383],
       [3082748.17530581],
       [2885692.75777052],
       [3510400.35804197],
       [2906656.10006151],
       [2952775.45310169],
       [2697022.67715163],
       [3057592.16455663],
       [2906656.10006151],
       [2956968.12155988],
       [3116289.5229714 ],
       [2789261.38323198],
       [2941874.51511037],
       [2765782.43986607],
       [3003506.74144588],
       [3078555.50684762],
       [2785907.24846542],
       [3141445.53372058],
       [2951936.91941005],
       [2906656.10006151],
       [3147734.53640788],
       [3070170.16993122],
       [2864729.41547954],
       [3030339.81957834],
       [2990509.46922547],
       [2812740.32659789],
       [2903301.96529495],
       [2977092.93015923],
       [2990509.46922547],
       [3030926.79316249],
       [3158216.20755337],
       [2906656.10006151],
       [2906656.10006151],
       [2952775.45310169],
       [2936843.31296054],
       [2946905.71726021],
       [2906656.10006151],
       [2906656.10006151],
       [2906656.10006151],
       [2906656.10006151],
```

```
[2956968.12155988],
[2764105.37248279],
[3027404.95165761],
[3027404.95165761],
[2933908.4450398 ],
[2906656.10006151],
[2835380.73627215],
[2906656.10006151],
[2906656.10006151],
[3362818.42831342],
[3158216.20755337],
[3047529.76025695],
[2906656.10006151],
[2906656.10006151],
[3148153.8032537 ],
[2926780.90866086],
[2936843.31296054],
[3340597.28548497],
[3148153.8032537 ],
[3510400.35804197],
[3049206.82764023],
[2906656.10006151],
[3417323.11826999],
[2738949.36173361],
[2906656.10006151],
[2824479.79828084],
[2956968.12155988],
[2742303.49650016],
[2760751.23771623],
[2941874.51511037],
[2948582.78464349],
[2881500.08931233],
[2906656.10006151],
[2906656.10006151],
[2738949.36173361],
[3283996.2612993 ],
[2906656.10006151],
[2718824.55313426],
[3095326.18068041],
[2962837.85740136],
[2735595.22696705],
[3024889.35058269],
[3122997.79250451],
[2822802.73089756],
[2969546.12693448],
[2806032.05706477],
[3007280.14305826],
```

```
[2906656.10006151],
[2747334.69865   ],
[3158216.20755337],
[2940197.44772709],
[2956968.12155988],
[2906656.10006151],
[2956968.12155988],
[2864729.41547954],
[2864729.41547954],
[2936004.7792689 ],
[2864729.41547954],
[2780876.04631558],
[2860536.74702134],
[2941874.51511037],
[2675220.801169  ],
[2958225.92209734],
[2956968.12155988],
[3105556.29171841],
[2764105.37248279],
[3210205.29643502],
[2973738.79539268],
[3074362.83838942],
[2982124.13230907],
[2713793.35098442],
[2941874.51511037],
[2992186.53660874],
[2951936.91941005],
[3009879.59750234],
[2927954.85582916],
[3017342.54735793],
[2950679.11887259],
[3711648.44403546],
[3003926.0082917 ],
[2948582.78464349],
[2864729.41547954],
[3364495.4956967 ],
[2806032.05706477],
[2892233.32056531],
[2839573.40473035],
[2806032.05706477],
[2990509.46922547],
[2906656.10006151],
[2856344.07856314],
[2792615.51799854],
[2822802.73089756],
[2936843.31296054],
[2889885.42622872],
```

```
[2961999.32370972],
[3283996.2612993 ],
[2806032.05706477],
[2797646.72014837],
[2822802.73089756],
[3283996.2612993 ],
[2864729.41547954],
[2936843.31296054],
[2956968.12155988],
[2834206.78910386],
[2772490.70939919],
[2856344.07856314],
[2680252.00331884],
[2709600.68252622],
[2915041.43697791],
[2982124.13230907],
[2639750.82601265],
[3072685.77100614],
[2667673.99794425],
[2924265.30758594],
[2915041.43697791],
[2956968.12155988],
[2975835.12962177],
[2966191.99216792],
[2944390.11618529],
[3057592.16455663],
[2789261.38323198],
[2760751.23771623],
[2951936.91941005],
[2864729.41547954],
[3264626.13302243],
[3107904.186055  ],
[2847958.74164675],
[2722178.68790082],
[3225298.90288453],
[3117966.59035467],
[2910848.76851971],
[2997637.0056044 ],
[2670189.59901917],
[2780876.04631558],
[3007280.14305826],
[2689475.87392687],
[3072685.77100614],
[2655095.99256965],
[2655095.99256965],
[3360302.8272385 ],
[2915041.43697791],
```

```
[2883177.15669561],
[2700376.81191819],
[3040821.49072384],
[3300766.93513209],
[2956968.12155988],
[2806032.05706477],
[3086940.84376401],
[2773329.24309083],
[3048032.88047194],
[2638325.31873686],
[2902295.72486499],
[2755720.0355664 ],
[2782553.11369886],
[2746915.43180418],
[2749011.76603328],
[2856344.07856314],
[2803516.45598985],
[2931812.1108107 ],
[2889885.42622872],
[2655095.99256965],
[2652580.39149473],
[2967030.52585956],
[2793118.63821352],
[3485244.34729279],
[2690314.40761851],
[2822383.46405174],
[2768298.04094099],
[2752365.90079984],
[2910010.23482807],
[2978937.70428084],
[2807289.85760223],
[2990509.46922547],
[3082748.17530581],
[2690314.40761851],
[3172135.86683459],
[2933573.03156314],
[3262194.38531667],
[2943551.58249365],
[2837057.80365543],
[2906656.10006151],
[2707923.61514294],
[3214146.40478573],
[2856344.07856314],
[2765782.43986607],
[2717566.7525968 ],
[2752365.90079984],
[2728886.95743393],
```

```
[2879823.02192905],
[2644195.05457834],
[2823641.2645892 ],
[2781714.58000722],
[2738949.36173361],
[2725532.82266737],
[2718824.55313426],
[2708762.14883458],
[2617361.97644587],
[2849635.80903002],
[2852989.94379658],
[2698699.74453491],
[3107904.186055  ],
[2747334.69865   ],
[2821964.19720592],
[2697861.21084327],
[2692830.00869343],
[3230330.10503437],
[2698699.74453491],
[2781714.58000722],
[2897012.96260766],
[2738949.36173361],
[3095326.18068041],
[2742303.49650016],
[2936843.31296054],
[2668680.23837421],
[2697861.21084327],
[2717986.01944262],
[2736266.05392036],
[2814417.39398116],
[2645033.58826998],
[2812740.32659789],
[2816094.46136444],
[2818610.06243936],
[2730564.02481721],
[2780876.04631558],
[2563276.55333512],
[2745238.3644209 ],
[2697022.67715163],
[2944390.11618529],
[2741632.66954685],
[2772490.70939919],
[3272256.78961635],
[2688637.34023523],
[2936843.31296054],
[2936843.31296054],
[2780876.04631558],
```

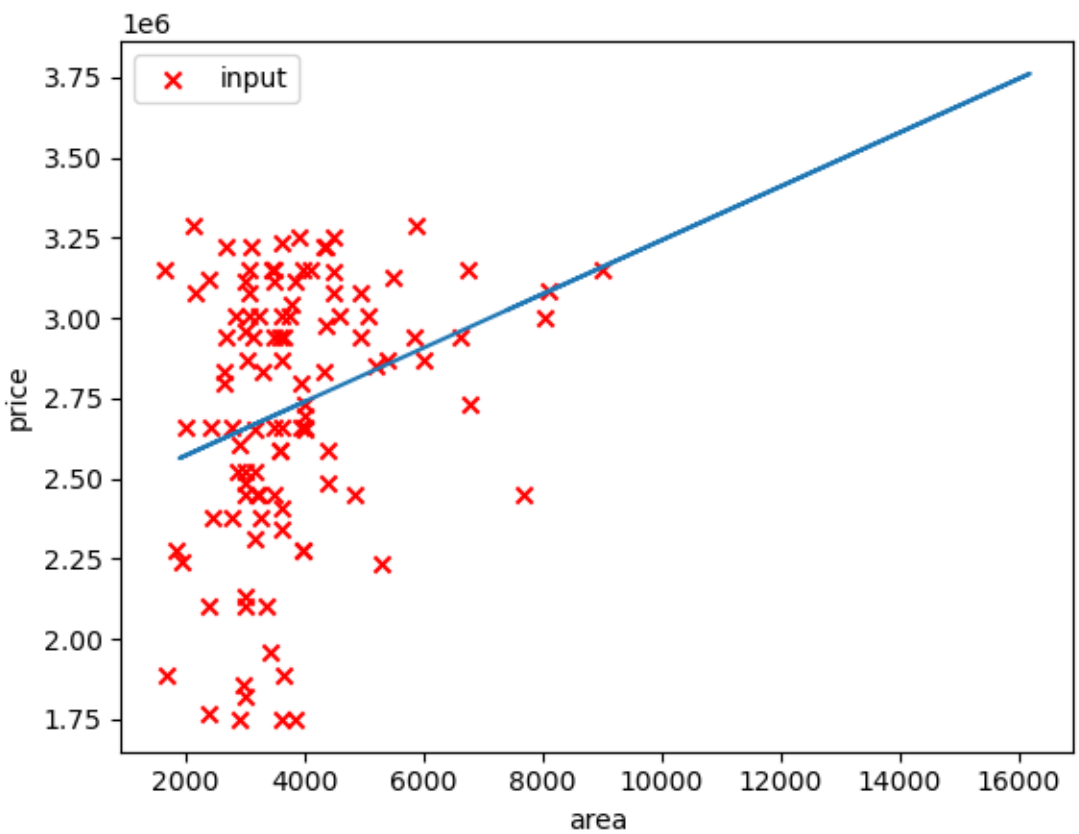
```
[2585916.96300939],
[2769136.57463263],
[3055076.56348171],
[2961160.79001808],
[2637235.22493773],
[2864729.41547954],
[2826156.86566412],
[2894078.09468692],
[2622393.17859571],
[2651154.88421895],
[2633881.09017117],
[2773329.24309083],
[2738949.36173361],
[2598494.96838399],
[2789261.38323198],
[2708762.14883458],
[2889885.42622872],
[2990509.46922547],
[2745573.77789756],
[2698699.74453491],
[2583401.36193447],
[2780876.04631558],
[3095326.18068041],
[2692830.00869343],
[2809386.19183133],
[2745657.63126672],
[2742806.61671515],
[2791944.69104522],
[2905398.29952405],
[2911687.30221135],
[2705408.01406803],
[2712116.28360114],
[2742303.49650016],
[2873114.75239593],
[2898270.76314512],
[2822131.90394425],
[2767459.50724935],
[2655095.99256965],
[2765782.43986607],
[2707923.61514294],
[2693668.54238507],
[2856344.07856314],
[2780876.04631558],
[2693668.54238507],
[2747334.69865   ],
[2946905.71726021],
[2780876.04631558],
```

```
[2735595.22696705],
[2743142.0301918 ],
[3012311.34520809],
[2864729.41547954],
[2655095.99256965],
[2679413.4696272 ],
[2723520.34180744],
[3081071.10792254],
[2583401.36193447],
[2720501.62051754],
[2670189.59901917],
[2847958.74164675],
[2670189.59901917],
[3003255.18133839],
[2745657.63126672],
[2726371.35635901],
[2572500.42394316],
[2586000.81637856],
[2684444.67177704],
[2667673.99794425],
[2807709.12444805],
[2690314.40761851],
[2705408.01406803],
[2892401.02730364],
[2643021.10741004],
[3107904.186055  ],
[3095326.18068041],
[2614846.37537096],
[2984639.73338399],
[2695345.60976835],
[2705408.01406803],
[2742303.49650016],
[2908333.16744479],
[2743142.0301918 ],
[2704066.3601614 ],
[2665158.39686933],
[2860536.74702134],
[2707923.61514294],
[2707923.61514294],
[2876468.88716249],
[2705408.01406803],
[2762428.30509951],
[2702892.41299311],
[2670189.59901917],
[2655095.99256965],
[2698699.74453491],
[2903301.96529495],
```

```
[2749850.29972492],
[2642517.98719506],
[2594302.29992579],
[2698699.74453491],
[2780876.04631558],
[2738949.36173361],
[2667673.99794425],
[2780876.04631558],
[2780876.04631558],
[2708762.14883458],
[2726371.35635901],
[2759074.17033296],
[2709600.68252622],
[2789261.38323198],
[2582562.82824283],
[2658114.71385956],
[2738110.82804197],
[3026063.29775098],
[2695345.60976835],
[2705408.01406803],
[2708762.14883458],
[2898270.76314512],
[2665158.39686933],
[3019858.14843285],
[2698028.9175816 ],
[3200142.89213535],
[2896593.69576184],
[3488933.895536  ],
[2814417.39398116],
[2660127.19471949],
[2849635.80903002],
[2583401.36193447],
[2738949.36173361],
[2670608.86586498],
[2726371.35635901],
[2583401.36193447],
[2622393.17859571],
[2567049.9549475 ],
[2742303.49650016],
[2804774.25652731],
[2692830.00869343],
[2708762.14883458],
[2697022.67715163],
[2819448.596131  ],
[2749011.76603328],
[2801839.38860657],
[2715470.4183677 ],
```

```
       [2717986.01944262],
       [2663481.32948605],
       [2670608.86586498],
       [2629939.98182047],
       [2583401.36193447],
       [2742303.49650016],
       [2803935.72283567],
       [2613169.30798768],
       [2670189.59901917],
       [2911687.30221135],
       [2695345.60976835],
       [2721507.8609475 ],
       [2742303.49650016]])
```

[19]:
```
plt.scatter(x_train,y_train,c='red',marker='x',label="input")#label used to␣
 ↪tell 'x' sybolises input
plt.plot(x_test,y_pred)
plt.xlabel("area")
plt.ylabel("price")
plt.legend()#to show label
plt.show()
```



[20]:
```
from sklearn.metrics import mean_squared_error
```

[21]:
```
mse = mean_squared_error(y_test,y_pred)
mse
```

[21]: 8574249538550.049

# linear_regression_mse

April 26, 2026

Problem Number 5:-

Implement Linear regression on a dataset and evaluate cost function and gradient descent.

Name - Aditya Kumar Garg

University Roll No- 2023827

[1]:
```
import numpy as np
```

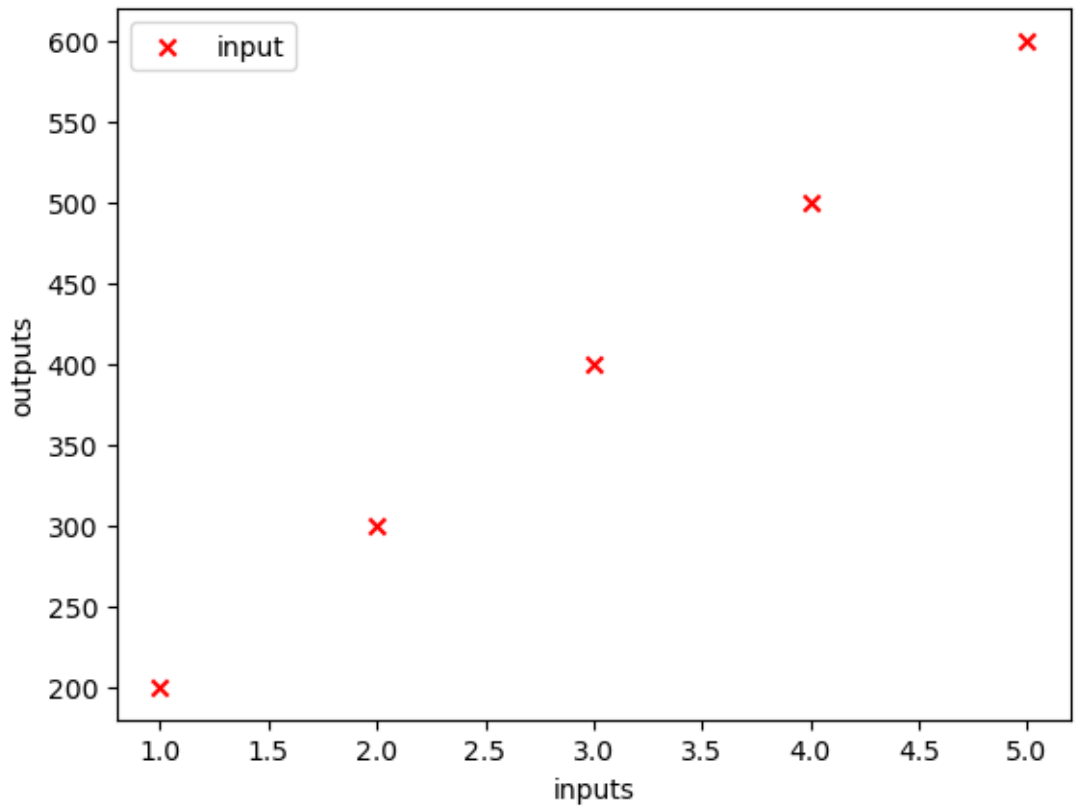
[2]:
```
x=np.array([1,2,3,4,5])
y=np.array([200,300,400,500,600])
```

[3]:
```
#show all training examples
for i in range(len(x)):
    print(f"(x[{i}],y[{i}])=({x[i]},{y[i]})")
```

```
(x[0],y[0])=(1,200)
(x[1],y[1])=(2,300)
(x[2],y[2])=(3,400)
(x[3],y[3])=(4,500)
(x[4],y[4])=(5,600)
```

[4]:
```
import matplotlib.pyplot as plt
```

[5]:
```
#scatter plot
plt.scatter(x,y,c='red',marker='x',label="input")#label used to tell 'x'␣
 ↪sybolises input
plt.xlabel("inputs")
plt.ylabel("outputs")
plt.legend()#to show label
plt.show()
```



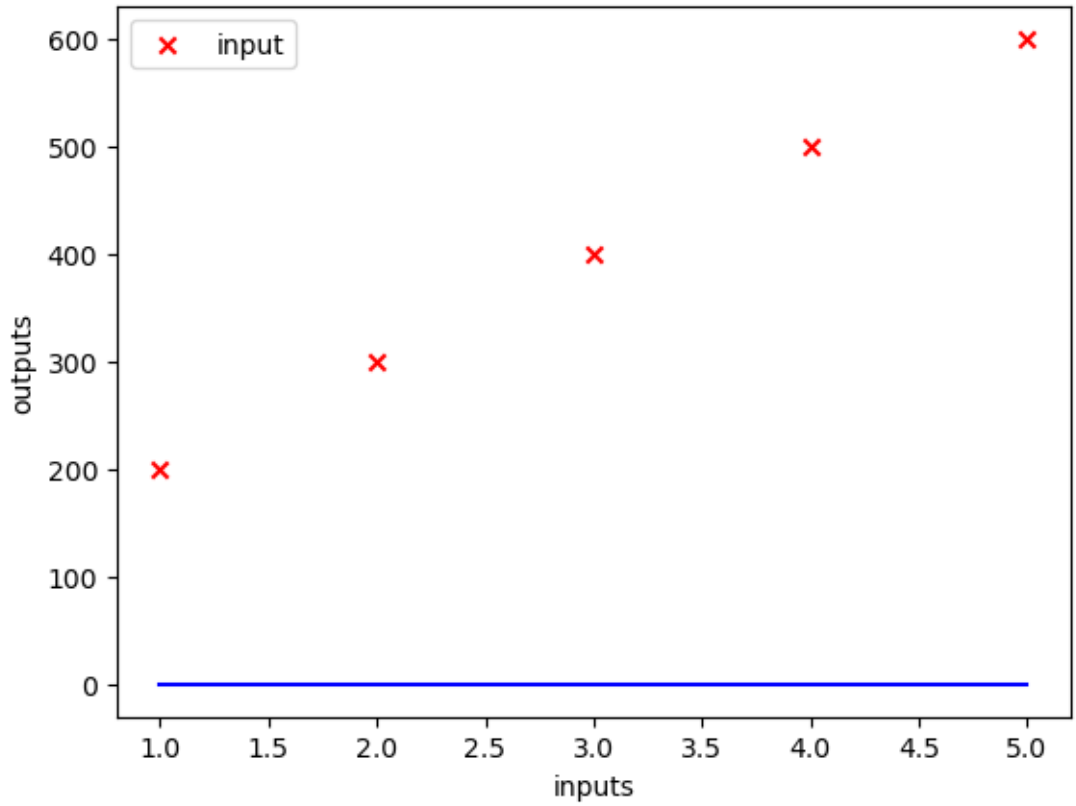

[6]:
```
#linear regression model              y=wx+b
def linear_regression_model(x,w,b):
    pred_y=np.zeros(len(x))
    for i in range(len(x)):
        pred_y[i]=w*x[i]+b
    return pred_y
```

[7]:
```
w=0
b=0
pred_y=linear_regression_model(x,w,b)
print(pred_y)
```

```
[0. 0. 0. 0. 0.]
```

[8]:
```
#plot linear regression model
plt.scatter(x,y,c='red',marker='x',label="input")#label used to tell 'x'␣
 ↪sybolises input
plt.plot(x,pred_y,c='blue')
plt.xlabel("inputs")
plt.ylabel("outputs")
plt.legend()
```

```
plt.show()
```




## 0.1 COST FUNCTION

[9]:
```
def cost_function_linear_regression(pred_y,y):
    sum_cost=0;
    for i in range(len(pred_y)):
        sum_cost=sum_cost+(pred_y[i]-y[i])**2
    cost_total= (1/2*(len(pred_y)))*sum_cost
    return cost_total
```

[10]:
```
mse=cost_function_linear_regression(pred_y,y)
```

[11]:
```
print(mse)
```

```
2250000.0
```

## 0.2 GRADIENT DESCENT

[12]:
```
# gradient descent = cost function - first order derivative with respect to w␣
↪and b
m=len(x)
def gradient_descent_cost_function(x,y,w,b):
    j_dw=0
    j_db=0
    for i in range (m):
        f_x = w*x[i]+b
        j_dw = j_dw + (f_x-y[i])*x[i]
        j_db = j_db + (f_x-y[i])
    j_dw = j_dw/m
    j_db = j_db/m
    return j_dw,j_db
```

[13]:
```
j_dw,j_db= gradient_descent_cost_function(x,y,w,b)
print(f"j_dw={j_dw} and j_db={j_db}")
```

```
j_dw=-1400.0 and j_db=-400.0
```

## 0.3 GRADIENT DESCENT ALGORITHM

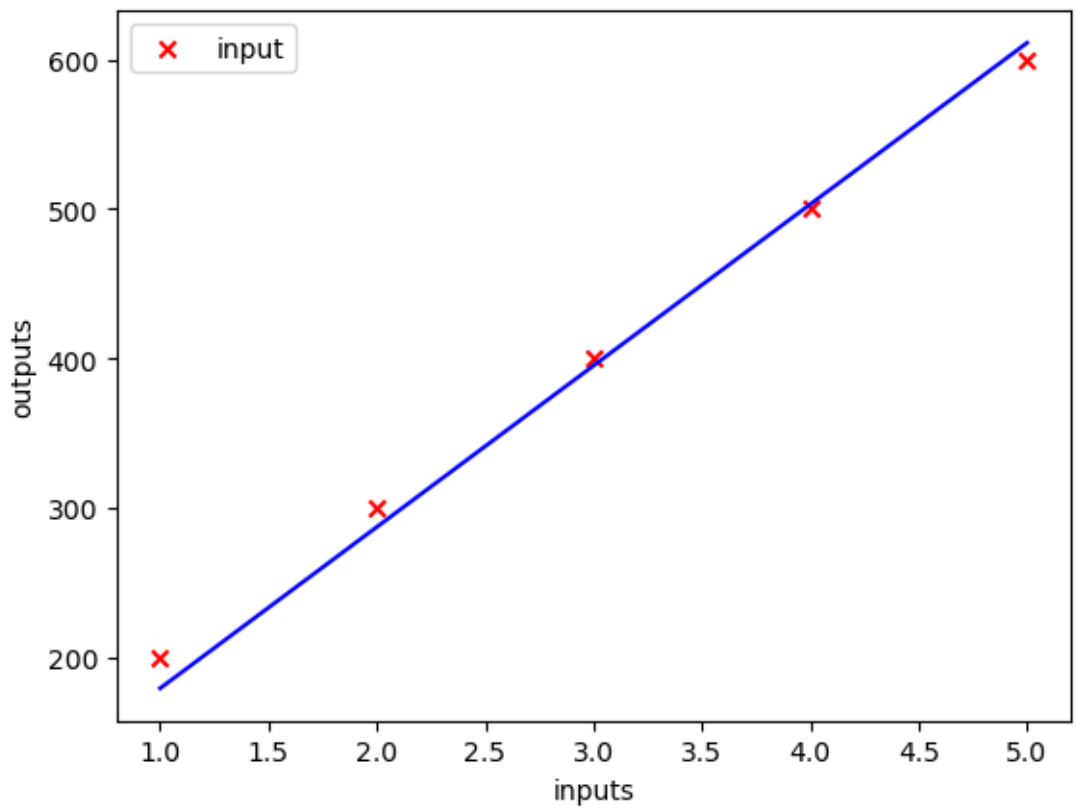
[14]:
```
def␣
↪gradient_descent_algorithm(x,y,w,b,learning_rate,gradient_descent_cost_function,iteration):
↪
    w,b=w,b
    for i in range (iteration):
        j_dw,j_db = gradient_descent_cost_function(x,y,w,b)
        w = w-learning_rate*j_dw
        b = b-learning_rate*j_db
    return w,b
```

[15]:
```
w=0
b=0
learning_rate=0.001
iteration=5000
w,b =␣
↪gradient_descent_algorithm(x,y,w,b,learning_rate,gradient_descent_cost_function,iteration)
print(f"w = {w} and b: {b}")
```

```
w = 107.9870315990104 and b: 71.16428165616972
```

[16]:
```
pred_y=linear_regression_model(x,w,b)
plt.scatter(x,y,c='red',marker='x',label="input")
plt.plot(x,pred_y,c='blue')
plt.xlabel("inputs")
plt.ylabel("outputs")
```

```
plt.legend()
plt.show()
```




[17]:
```
def gradient_descent_algo(x,y,w,b,j_dw,j_db,learning_rate,iteration):
    w,b=w,b
    for i in range (iteration):
        j_dw=0
        j_db=0
        for i in range (len(y)):
            f_x = w*x[i]+b
            j_dw = j_dw + (f_x-y[i])*x[i]
            j_db = j_db + (f_x-y[i])
        j_dw = j_dw/len(y)
        j_db = j_db/len(x)
        w = w - learning_rate*j_dw
        b = b - learning_rate*j_db
    return w,b
```

[18]:
```
w=0
b=0
learning_rate=0.001
```

```python
iteration=1000
w,b=gradient_descent_algo(x,y,w,b,j_dw,j_db,learning_rate,iteration)
print(f"w = {w} and b: {b}")
```

```
w = 115.70560348553238 and b: 43.29466213948348
```

# logistic_regression_sklearn

April 26, 2026

Problem Number 6:-

Implement logistic regression using Scikit-learn and evaluate using accuracy, precision, recall, and F1-score.

Name - Aditya Kumar Garg

University Roll No- 2023827

[1]:
```
import pandas as pd
```

[2]:
```
#load the data
data = pd.read_csv('Brain_Tumor.csv')
```

[3]:
```
data
```

[3]:
```
      Class  Tumor_Size Unnamed: 2
0         0   98.613971        NaN
1         0   63.858816        NaN
2         0   81.867206          s
3         1  151.229741        NaN
4         1  174.988756        NaN
…     …          …        …
3743      1  158.437600        NaN
3744      1  161.158675        NaN
3745      1  167.130118        NaN
3746      1  223.812932        NaN
3747      1  239.251388        NaN

[3748 rows x 3 columns]
```

[4]:
```
#Extract the input and output
x= data['Tumor_Size']
y= data['Class']
```

[5]:
```
# 2d-array
import numpy as np
x = np.array(x).reshape(-1,1)
#y = np.array(y).reshape(-1,1)
```

[6]:
```
print(f"Total inputs - {len(x)}")
print(f"Total outputs - {len(y)}")
```

```
Total inputs - 3748
Total outputs - 3748
```

[7]:
```
#split the dataset using train_test_split() method
from sklearn.model_selection import train_test_split
```

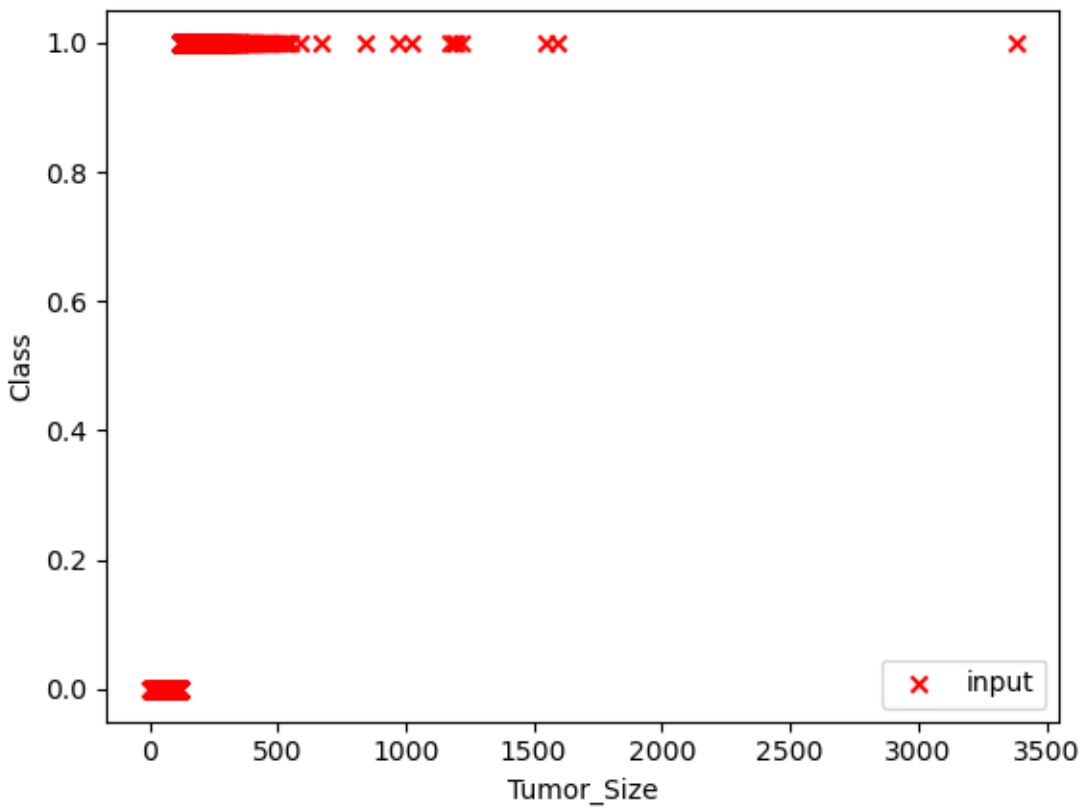
[8]:
```
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.2)
```

[9]:
```
print(f"Total Training inputs - {len(x_train)}")
print(f"Total Testing outputs - {len(x_test)}")
```

```
Total Training inputs - 2998
Total Testing outputs - 750
```

[10]:
```
import matplotlib.pyplot as plt
```

[11]:
```
plt.scatter(x_train,y_train,c='red',marker='x',label="input")#label used to␣
↪tell 'x' sybolises input
plt.xlabel("Tumor_Size")
plt.ylabel("Class")
plt.legend()#to show label
plt.show()
```




[12]:
```
# make logistic regression using sklearn
from sklearn.linear_model import LogisticRegression
```

[13]:
```
# object to LogisticRegression
model_logreg = LogisticRegression()
```

[14]:
```
# fit the model
model_logreg.fit(x_train, y_train)
```

[14]:
```
LogisticRegression()
```

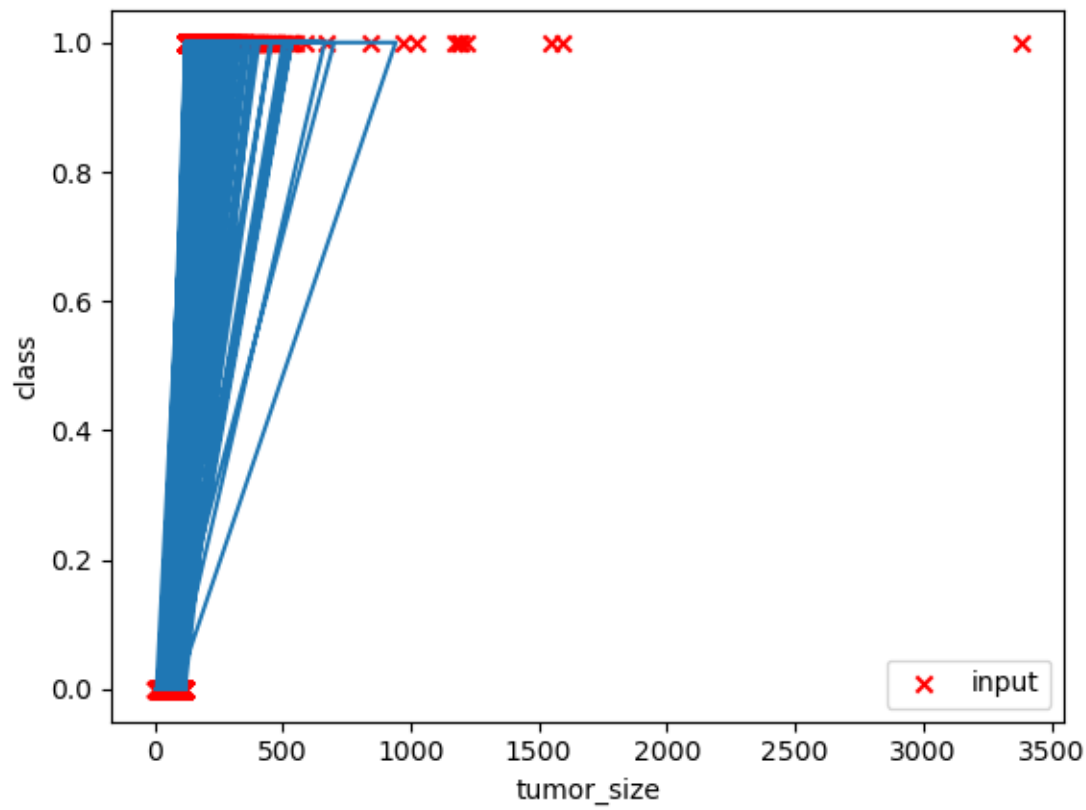
[15]:
```
y_pred = model_logreg.predict(x_test)
```

[16]:
```
y_pred
```

[16]:
```
array([0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0,
       0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0,
       1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0,
       0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1,
       0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0,
```

```
1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0,
0, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0,
0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0,
1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0,
0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1,
0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0,
1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0,
0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0,
0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0,
0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1,
1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0,
0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 1,
0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0,
1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0,
0, 1, 1, 1, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0,
1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1,
0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1,
0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1,
0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0,
0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1,
0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0,
1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1,
1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1,
0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1,
0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1,
1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
0, 0])
```

[17]:
```python
plt.scatter(x_train,y_train,c='red',marker='x',label="input")#label used to␣
↪tell 'x' sybolises input
plt.plot(x_test,y_pred)
plt.xlabel("tumor_size")
plt.ylabel("class")
plt.legend()#to show label
plt.show()
```




[18]:
```
from sklearn.metrics import accuracy_score,confusion_matrix,␣
↪classification_report
```

[19]:
```
print(f"accuracy of our proposed model:{accuracy_score(y_pred,y_test)}")
```

```
accuracy of our proposed model:1.0
```

[20]:
```
cm = confusion_matrix(y_pred,y_test)
print(f"confusion matrix{cm}")
```

```
confusion matrix[[447   0]
 [  0 303]]
```

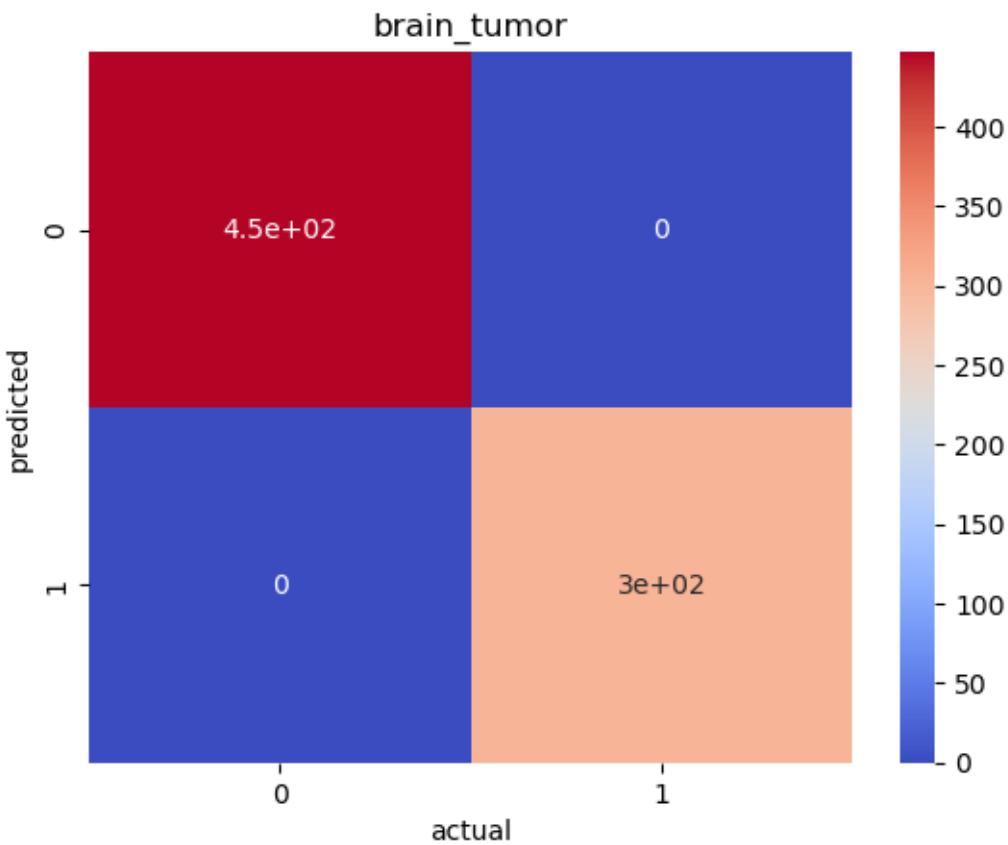
[21]:
```
output=['malignant','begnin']
print(f"{classification_report(y_pred,y_test,target_names=output)}")
```

```
              precision    recall  f1-score   support

   malignant       1.00      1.00      1.00       447
      begnin       1.00      1.00      1.00       303

    accuracy                           1.00       750
   macro avg       1.00      1.00      1.00       750
```

```
weighted avg       1.00      1.00      1.00       750
```

[22]:
```python
import seaborn as sns
```

[23]:
```python
sns.heatmap(cm,cmap='coolwarm', annot=True, xticklabels=[0,1],␣
↪yticklabels=[0,1])
plt.xlabel("actual")
plt.ylabel("predicted")
plt.title("brain_tumor")
plt.show()
```



# logistic_regression_dataset

April 26, 2026

Problem Number 7:-

Implement Logistic regression on a dataset and evaluate cost function and gradient descent.

Name - Aditya Kumar Garg

University Roll No- 2023827

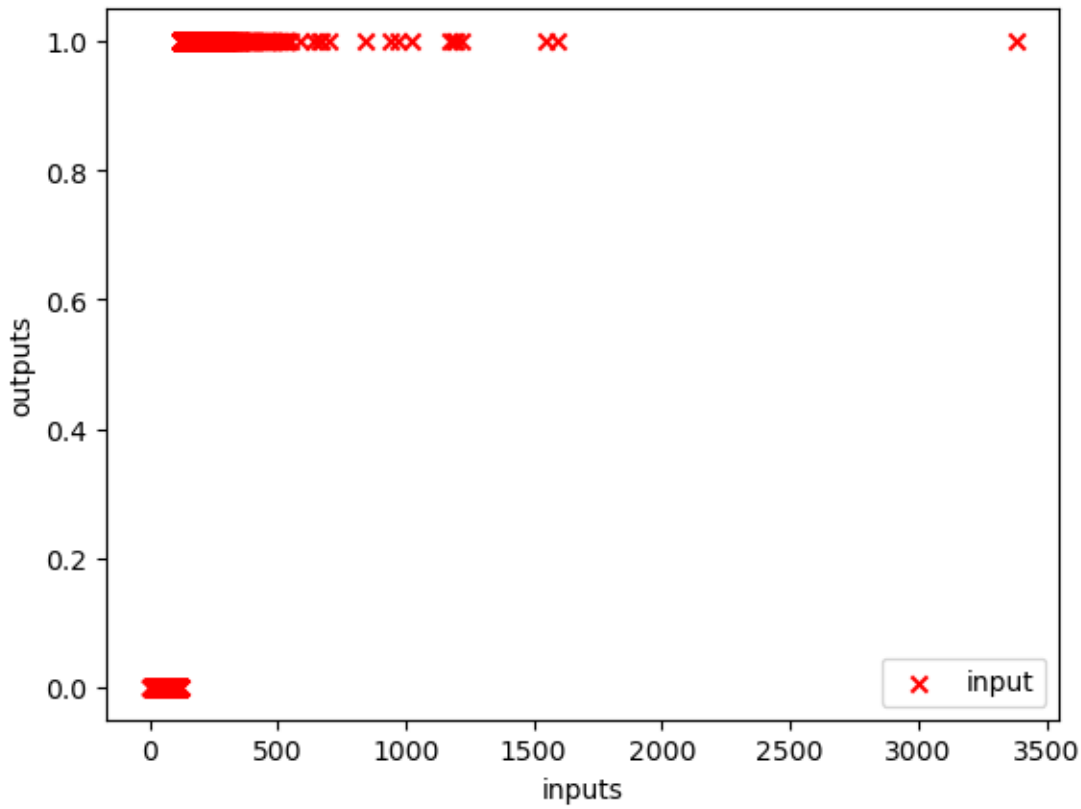
[1]:
```
import pandas as pd
```

[2]:
```
data = pd.read_csv('Brain_Tumor.csv')
```

[3]:
```
Input = data['Tumor_Size']
Output = data['Class']
```

[4]:
```
import matplotlib.pyplot as plt
```

[5]:
```
#scatter plot
plt.scatter(Input,Output,c='red',marker='x',label="input")#label used to tell␣
↪'x' sybolises input
plt.xlabel("inputs")
plt.ylabel("outputs")
plt.legend()#to show label
plt.show()
```



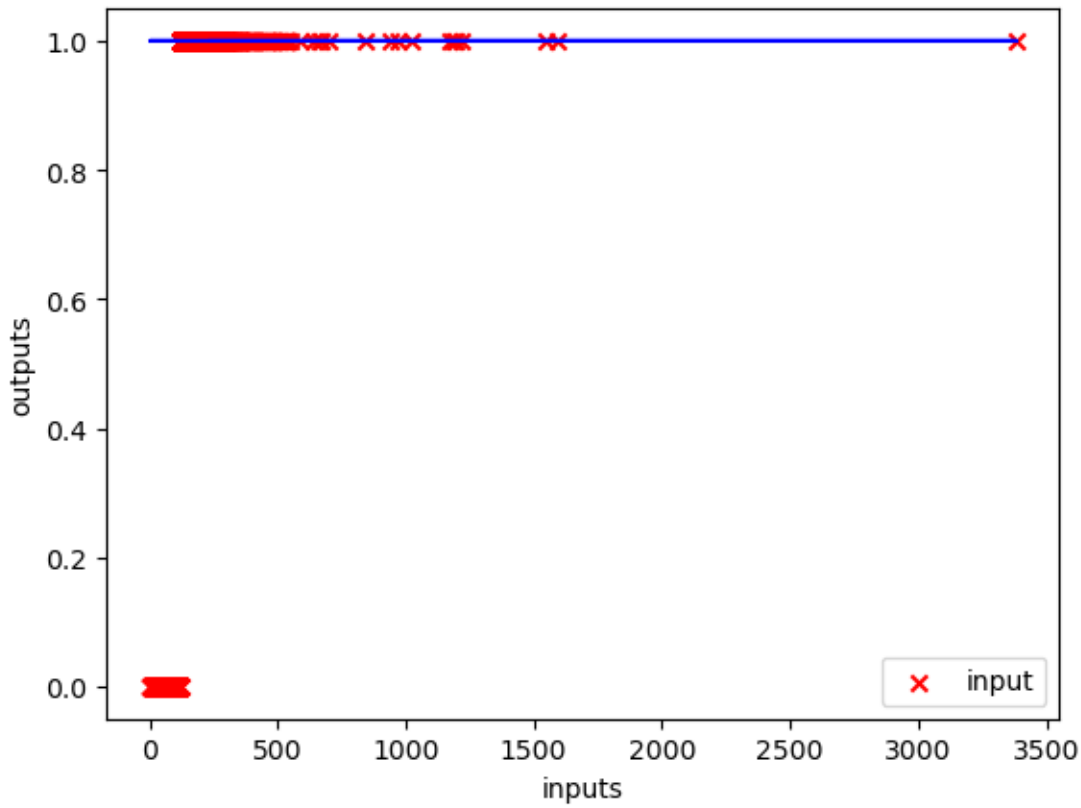

[6]:
```
#logistic regression model
import numpy as np
def logistic_regression_model(Input,w,b):
    pred_y=np.zeros(len(Input))
    for i in range(len(Input)):
        z = w * Input[i] + b
        pred_y[i] = 1 / (1 + np.exp(-z))
    return pred_y
```

[7]:
```
w=100
b=100
pred_y=logistic_regression_model(Input,w,b)
print(pred_y)
```

```
[1. 1. 1. … 1. 1. 1.]
```

[8]:
```
#plot linear regression model
plt.scatter(Input,Output,c='red',marker='x',label="input")#label used to tell␣
↪'x' sybolises input
plt.plot(Input,pred_y,c='blue')
plt.xlabel("inputs")
```

```
plt.ylabel("outputs")
plt.legend()
plt.show()
```




## 0.1 COST FUNCTION

[9]:
```
def sigmoid(x):
    return 1/(1+np.exp(-x))
```

[10]:
```
def cost_function_logistic_regression(pred_y,Output):
    sum_cost=0;
    for i in range(len(pred_y)):
        epsilon = 1e-8
        sum_cost=sum_cost+(Output[i]*np.
↪log(sigmoid(pred_y[i]+epsilon))+(1-Output[i])*(np.
↪log(1-sigmoid(pred_y[i]+epsilon))))
    cost_total= (-1/len(pred_y))*sum_cost
    return cost_total
```

[11]:
```
mse=cost_function_logistic_regression(pred_y,Output)
```

[12]:
```
print(mse)
```

```
0.8882350096820413
```

[13]:
```
class_counts=data['Class'].value_counts()
print(class_counts)
```

```
Class
0    2155
1    1593
Name: count, dtype: int64
```

## 0.2 Gradient Descent

[16]:
```
def gradient_descent(Input, Output, w, b, alpha, iterations):
    m = len(Output)
    for i in range(iterations):
        pred_y = sigmoid(w*Input[i] + b)
        dw = (1/m) * np.sum((pred_y - Output[i]) * Input[i])
        db = (1/m) * np.sum(pred_y - Output[i])
        w -= alpha * dw
        b -= alpha * db
    return w, b
```

[19]:
```
iterations=1000
alpha_values = [0.01, 0.1, 1]
for alpha in alpha_values:
    w = 0
    b = 0
    w,b = gradient_descent(Input, Output, w, b, alpha, iterations)
    print(f"w = {w} and b: {b}")
```

```
w = 0.0010079463853270823 and b: -0.000568101978699987
w = -0.004741583407882011 and b: -0.005495780402839173
w = -0.02249821003051352 and b: -0.053309404061453396
```

# KNN_sklearn

April 26, 2026

Problem Number 8:-

Implement and evaluate k-NN for classification and regression tasks.

Name - Aditya Kumar Garg

University Roll No - 2023827

[1]:
```
import pandas as pd
```

[2]:
```
df = pd.read_csv("KNN_Cancer_Data.csv")
```

[3]:
```
X = df[['radius_mean','texture_mean']]
y = df['diagnosis']
```

[4]:
```
from sklearn.model_selection import train_test_split
```

[5]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4)
```

[6]:
```
print(f"Total Training inputs - {len(X_train)}")
print(f"Total Testing outputs - {len(X_test)}")
```

```
Total Training inputs - 341
Total Testing outputs - 228
```

[7]:
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[8]:
```
from sklearn.neighbors import KNeighborsClassifier
```

[9]:
```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

[10]:
```
print(y_pred)
```

```
['B' 'B' 'M' 'B' 'M' 'B' 'B' 'M' 'B' 'B' 'B' 'B' 'M' 'B' 'B' 'B' 'M' 'M'
 'B' 'B' 'M' 'B' 'B' 'B' 'M' 'B' 'B' 'M' 'B' 'B' 'B' 'M' 'B' 'B' 'M' 'B'
 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'M' 'B' 'B' 'B' 'B' 'B' 'B' 'B' 'B'
```

```
 'B' 'M' 'M' 'M' 'B' 'B' 'M' 'B' 'M' 'B' 'B' 'B' 'M' 'M' 'B' 'B' 'B' 'B'
 'B' 'B' 'M' 'B' 'B' 'M' 'M' 'B' 'B' 'B' 'B' 'B' 'B' 'M' 'B' 'M' 'B' 'B'
 'B' 'B' 'B' 'B' 'M' 'B' 'B' 'B' 'M' 'M' 'B' 'B' 'B' 'M' 'B' 'M' 'M' 'M'
 'B' 'B' 'M' 'B' 'B' 'B' 'B' 'M' 'M' 'B' 'B' 'B' 'B' 'M' 'B' 'B' 'B' 'M'
 'M' 'M' 'B' 'B' 'B' 'B' 'B' 'M' 'B' 'B' 'B' 'M' 'B' 'B' 'M' 'B' 'B' 'B'
 'B' 'B' 'M' 'B' 'B' 'B' 'M' 'M' 'B' 'B' 'M' 'B' 'B' 'B' 'B' 'M' 'B' 'B'
 'M' 'B' 'M' 'B' 'B' 'M' 'B' 'B' 'B' 'B' 'B' 'B' 'M' 'B' 'M' 'B' 'B' 'B'
 'M' 'M' 'B' 'B' 'M' 'M' 'M' 'B' 'M' 'B' 'B' 'M' 'B' 'B' 'B' 'B' 'B' 'M'
 'B' 'B' 'B' 'M' 'B' 'B' 'B' 'B' 'M' 'M' 'B' 'M' 'M' 'B' 'B' 'M' 'M' 'B'
 'B' 'B' 'M' 'B' 'B' 'M' 'B' 'B' 'B' 'B' 'B' 'M']
```

[11]:
```
from sklearn.metrics import accuracy_score, classification_report,␣
↪confusion_matrix
```

[12]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.8947368421052632

Confusion Matrix:
 [[142   7]
 [ 17  62]]

Classification Report:
               precision    recall  f1-score   support

           B       0.89      0.95      0.92       149
           M       0.90      0.78      0.84        79

    accuracy                           0.89       228
   macro avg       0.90      0.87      0.88       228
weighted avg       0.89      0.89      0.89       228
```

[13]:
```
sample = pd.DataFrame(
    [[15.0, 20.0]]
)
prediction = knn.predict(sample)
print("Prediction for sample:", prediction)
```

```
Prediction for sample: ['M']
```

# Naive_Bayes

April 26, 2026

Problem Number 9:-

Implement Naïve Bayes for classification. Train and evaluate the model using labelled dataset.

Name - Aditya Kumar Garg

University Roll No- 2023827

[1]:
```
import numpy as np
```

[2]:
```
import pandas as pd
```

[3]:
```
data=pd.read_csv('play_tennis.csv')
```

[4]:
```
data=data.drop(['day'],axis=1)
```

[5]:
```
data.head()
```

[5]:

| | outlook | temp | humidity | wind | play |
|---|---|---|---|---|---|
| 0 | Sunny | Hot | High | Weak | No |
| 1 | Sunny | Hot | High | Strong | No |
| 2 | Overcast | Hot | High | Weak | Yes |
| 3 | Rain | Mild | High | Weak | Yes |
| 4 | Rain | Cool | Normal | Weak | Yes |

[6]:
```
from sklearn import preprocessing
process_tennis=preprocessing.LabelEncoder()
data = data.apply(process_tennis.fit_transform)
```

[7]:
```
X=data.drop(['play'],axis=1)
Y=data['play']
```

[8]:
```
print(X)
```

```
   outlook  temp  humidity  wind
0        2     1         0     1
1        2     1         0     0
2        0     1         0     1
3        1     2         0     1
4        1     0         1     1
5        1     0         1     0
```

```
6          0      0          1      0
7          2      2          0      1
8          2      0          1      1
9          1      2          1      1
10         2      2          1      0
11         0      2          0      0
12         0      1          1      1
13         1      2          0      0
```

[9]:
```
print(Y)
```

```
0     0
1     0
2     1
3     1
4     1
5     0
6     1
7     0
8     1
9     1
10    1
11    1
12    1
13    0
Name: play, dtype: int64
```

[10]:
```
from sklearn.model_selection import train_test_split
```

[11]:
```
x_train, x_test, y_train, y_test = train_test_split(X,Y,test_size=0.2)
```

[12]:
```
from sklearn.naive_bayes import GaussianNB
```

[13]:
```
model = GaussianNB()
```

[14]:
```
model.fit(x_train,y_train)
```

[14]:
```
GaussianNB()
```

[15]:
```
from sklearn.metrics import accuracy_score, confusion_matrix,␣
↪classification_report
```

[16]:
```
y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print("Classification Report:\n", classification_report(y_test, y_pred,␣
↪zero_division=0))
```

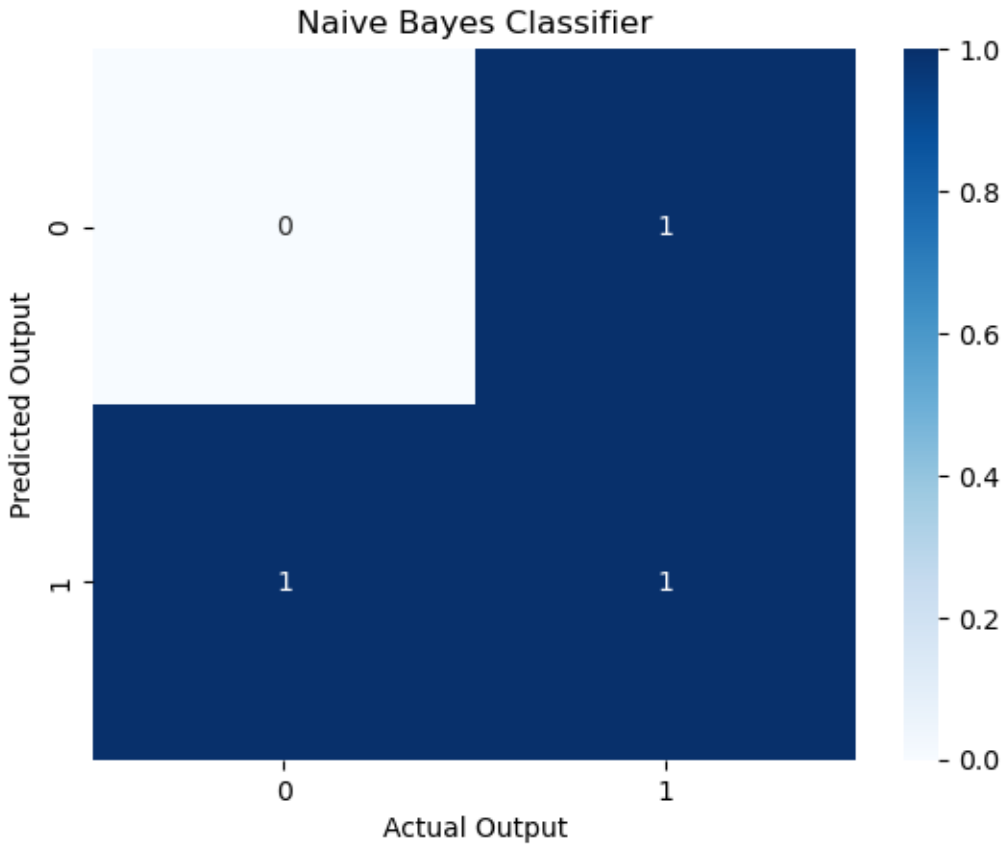
```
Accuracy: 0.3333333333333333
Confusion Matrix:
 [[0 1]
 [1 1]]
Classification Report:
               precision    recall  f1-score   support

           0       0.00      0.00      0.00         1
           1       0.50      0.50      0.50         2

    accuracy                           0.33         3
   macro avg       0.25      0.25      0.25         3
weighted avg       0.33      0.33      0.33         3
```

[17]:
```python
print(y_pred)
```

```
[1 0 1]
```

[18]:
```python
import matplotlib.pyplot as plt
import seaborn as sns
```

[19]:
```python
sns.heatmap(cm,cmap='Blues', annot=True, xticklabels=[0,1], yticklabels=[0,1])
plt.xlabel('Actual Output')
plt.ylabel('Predicted Output')
plt.title('Naive Bayes Classifier')
plt.show()
```


Naive Bayes Classifier
Predicted Output
Actual Output
0
1
0
1
0
1
1.0
0.8
0.6
0.4
0.2
0.0

# Support_Vector_Machine

April 26, 2026

Problem Number 10:-

Implement SVM for classification and train SVM with linear and kernel-based approaches.

Name - Aditya Kumar Garg

University Roll No - 2023827

[1]:
```
import pandas as pd
```

[2]:
```
data = pd.read_csv("mouse_viral_study.csv")
```

[3]:
```
data.head()
```

[3]:

| | Med_1_mL | Med_2_mL | Virus Present |
|---|---|---|---|
| 0 | 6.508231 | 8.582531 | 0 |
| 1 | 4.126116 | 3.073459 | 1 |
| 2 | 6.427870 | 6.369758 | 0 |
| 3 | 3.672953 | 4.905215 | 1 |
| 4 | 1.580321 | 2.440562 | 1 |

[4]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 3 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Med_1_mL       400 non-null    float64
 1   Med_2_mL       400 non-null    float64
 2   Virus Present  400 non-null    int64
dtypes: float64(2), int64(1)
memory usage: 9.5 KB
```

[5]:
```
X=data.drop('Virus Present',axis=1)
Y=data['Virus Present']
```

[6]:
```
print(X)
```

```
     Med_1_mL  Med_2_mL
0    6.508231  8.582531
```

```
1    4.126116  3.073459
2    6.427870  6.369758
3    3.672953  4.905215
4    1.580321  2.440562
..        …       …
395  2.884122  3.271748
396  7.290855  9.488672
397  7.895325  8.272529
398  2.690592  2.674979
399  7.587821  9.473968

[400 rows x 2 columns]
```

[7]:
```python
print(Y)
```

```
0      0
1      1
2      0
3      1
4      1
      ..
395    1
396    0
397    0
398    1
399    0
Name: Virus Present, Length: 400, dtype: int64
```

[8]:
```python
from sklearn.model_selection import train_test_split
```

[9]:
```python
x_train, x_test, y_train, y_test = train_test_split(X,Y,test_size=0.2)
```

[10]:
```python
from sklearn.svm import SVC
```

[11]:
```python
# Linear Kernel
model = SVC(kernel='linear')
model.fit(X,Y)
y_pred = model.predict(x_test)
print(y_pred)
```

```
[0 0 1 0 1 0 0 1 0 1 0 0 0 0 0 0 1 0 1 1 0 0 0 0 1 0 1 0 1 0 1 1 1 1 1 1 1
 0 1 1 0 1 0 1 1 0 0 1 0 0 1 1 1 1 1 0 1 0 0 0 1 0 0 0 1 0 0 1 0 1 0 0 1 0
 1 0 0 0 1 0]
```

[12]:
```python
# Kernel based - rbf
svm_rbf = SVC(kernel='rbf')
svm_rbf.fit(x_train, y_train)
y_pred_rbf = svm_rbf.predict(x_test)
print(y_pred_rbf)
```

```
[0 0 1 0 1 0 0 1 0 1 0 0 0 0 0 0 1 0 1 1 0 0 0 0 1 0 1 0 1 0 1 1 1 1 1 1 1
 0 1 1 0 1 0 1 1 0 0 1 0 0 1 1 1 1 1 0 1 0 0 0 1 0 0 0 1 0 0 1 0 1 0 0 1 0
 1 0 0 0 1 0]
```

[13]:
```
from sklearn.metrics import accuracy_score, confusion_matrix,␣
↪classification_report
```

[14]:
```
# Linear Kernel
print("Accuracy:", accuracy_score(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00        44
           1       1.00      1.00      1.00        36

    accuracy                           1.00        80
   macro avg       1.00      1.00      1.00        80
weighted avg       1.00      1.00      1.00        80
```

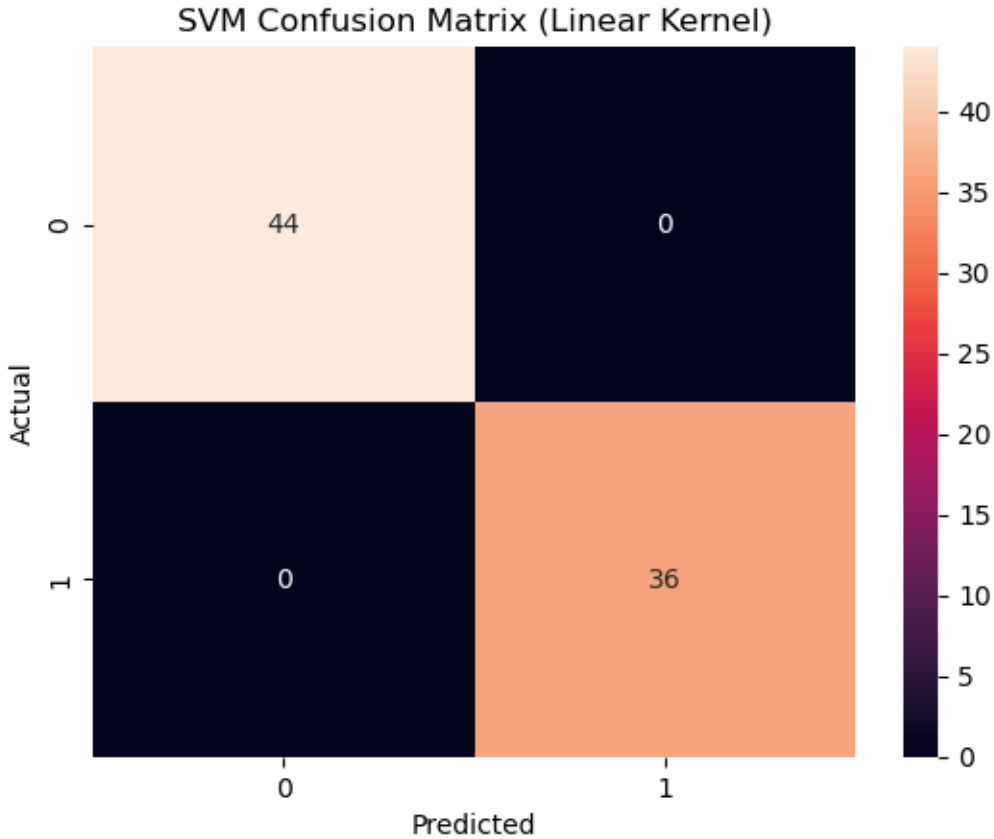
[15]:
```
# RBF Kernel
print("Accuracy:", accuracy_score(y_test, y_pred_rbf))
cm = confusion_matrix(y_test, y_pred_rbf)
print("Classification Report:\n", classification_report(y_test, y_pred_rbf))
```

```
Accuracy: 1.0
Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00        44
           1       1.00      1.00      1.00        36

    accuracy                           1.00        80
   macro avg       1.00      1.00      1.00        80
weighted avg       1.00      1.00      1.00        80
```

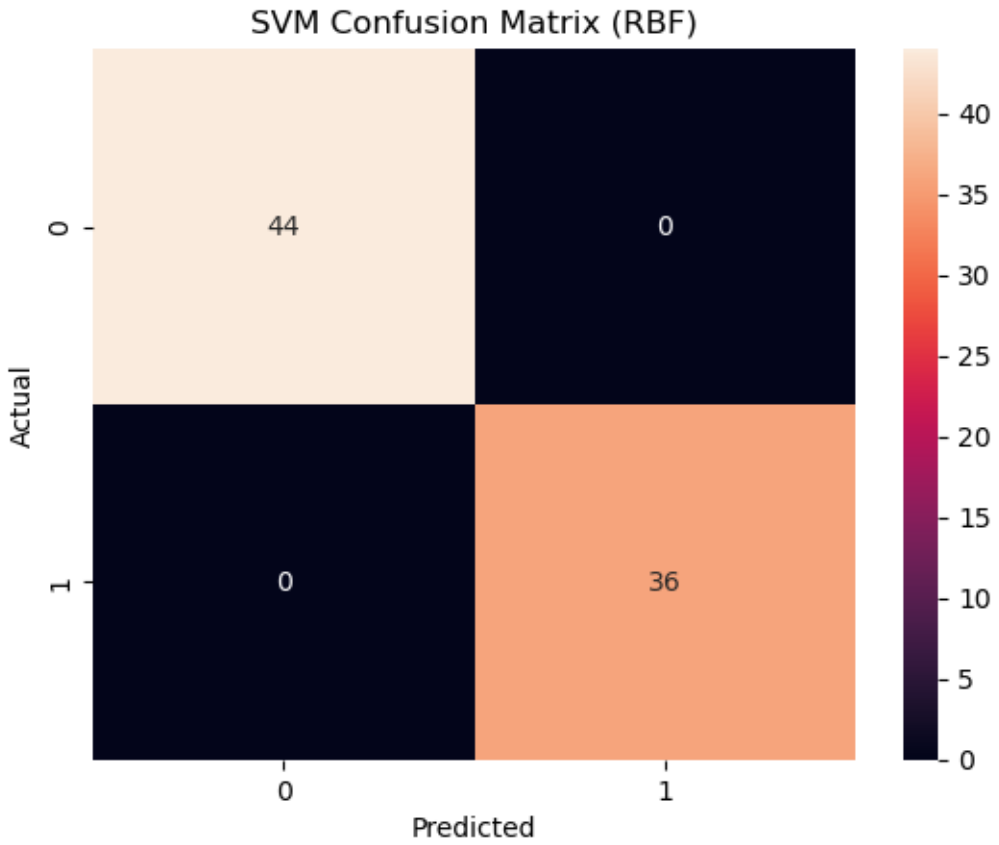
[16]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

[17]:
```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("SVM Confusion Matrix (Linear Kernel)")
```

```
plt.show()
```




[18]:
```
cm = confusion_matrix(y_test, y_pred_rbf)
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("SVM Confusion Matrix (RBF)")
plt.show()
```


SVM Confusion Matrix (RBF)
44
0
0
36
Actual
0
1
Predicted
0
1
40
35
30
25
20
15
10
5
0

# Decision_tree

April 26, 2026

Problem Number 11:-

Implement Decision Tree for classification. Train and evaluate the model using labelled dataset.

Name - Aditya Kumar Garg

University Roll No- 2023827

[1]:
```
import numpy as np
import pandas as pd
```

[2]:
```
data=pd.read_csv('play_tennis.csv')
```

[3]:
```
data=data.drop(['day'],axis=1)
```

[4]:
```
data.head()
```

[4]:

| | outlook | temp | humidity | wind | play |
|---|---|---|---|---|---|
| 0 | Sunny | Hot | High | Weak | No |
| 1 | Sunny | Hot | High | Strong | No |
| 2 | Overcast | Hot | High | Weak | Yes |
| 3 | Rain | Mild | High | Weak | Yes |
| 4 | Rain | Cool | Normal | Weak | Yes |

[5]:
```
from sklearn import preprocessing
process_tennis=preprocessing.LabelEncoder()
data = data.apply(process_tennis.fit_transform)
```

[6]:
```
X=data.drop(['play'],axis=1)
Y=data['play']
```

[7]:
```
print(X)
print(Y)
```

```
   outlook  temp  humidity  wind
0        2     1         0     1
1        2     1         0     0
2        0     1         0     1
3        1     2         0     1
4        1     0         1     1
5        1     0         1     0
```

```
6          0      0          1      0
7          2      2          0      1
8          2      0          1      1
9          1      2          1      1
10         2      2          1      0
11         0      2          0      0
12         0      1          1      1
13         1      2          0      0
0     0
1     0
2     1
3     1
4     1
5     0
6     1
7     0
8     1
9     1
10    1
11    1
12    1
13    0
Name: play, dtype: int64
```

[8]:
```
from sklearn.model_selection import train_test_split
```

[9]:
```
x_train, x_test, y_train, y_test = train_test_split(X,Y,test_size=0.2)
```

[10]:
```
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
```

[11]:
```
model = DecisionTreeClassifier()
```

[12]:
```
model.fit(x_train,y_train)
```

[12]:
```
DecisionTreeClassifier()
```

[13]:
```
from sklearn.metrics import accuracy_score, confusion_matrix,␣
↪classification_report
```

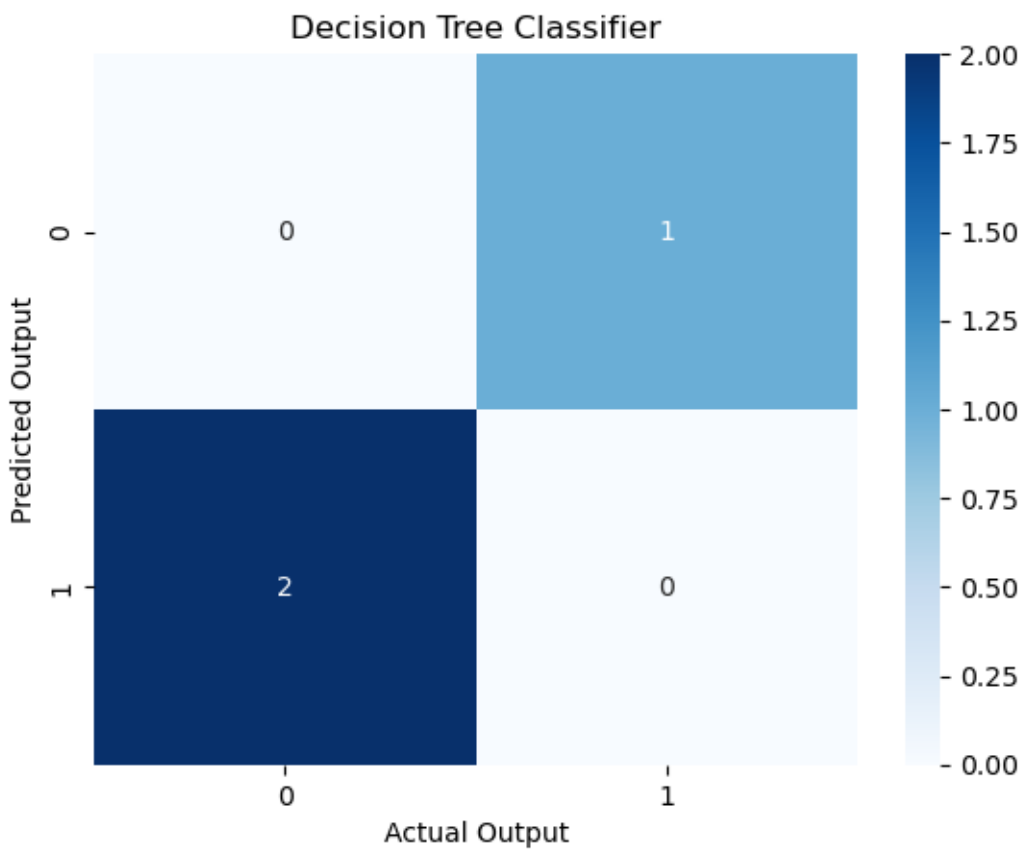
[14]:
```
y_pred = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print("Classification Report:\n", classification_report(y_test, y_pred))
```

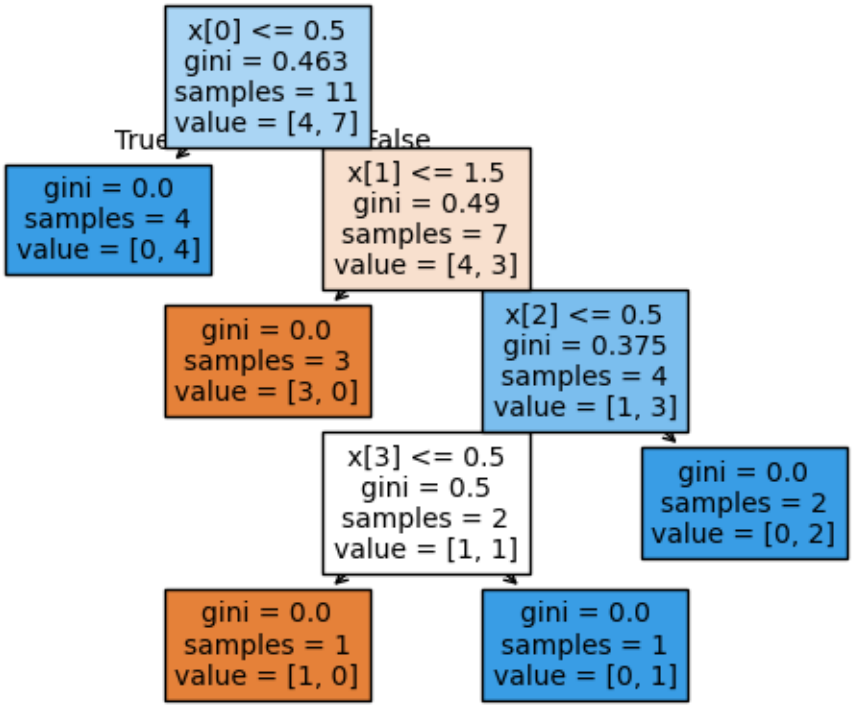
```
Accuracy: 0.0
Confusion Matrix:
 [[0 1]
```

```
 [2 0]]
Classification Report:
              precision    recall  f1-score   support

           0       0.00      0.00      0.00       1.0
           1       0.00      0.00      0.00       2.0

    accuracy                           0.00       3.0
   macro avg       0.00      0.00      0.00       3.0
weighted avg       0.00      0.00      0.00       3.0
```

[15]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

[17]:
```
sns.heatmap(cm,cmap='Blues', annot=True, xticklabels=[0,1], yticklabels=[0,1])
plt.xlabel('Actual Output')
plt.ylabel('Predicted Output')
plt.title('Decision Tree Classifier')
plt.show()
tree.plot_tree(model, filled=True)
plt.show()
```




x[0] <= 0.5
gini = 0.463
samples = 11
value = [4, 7]
True
False
gini = 0.0
samples = 4
value = [0, 4]
x[1] <= 1.5
gini = 0.49
samples = 7
value = [4, 3]
gini = 0.0
samples = 3
value = [3, 0]
x[2] <= 0.5
gini = 0.375
samples = 4
value = [1, 3]
x[3] <= 0.5
gini = 0.5
samples = 2
value = [1, 1]
gini = 0.0
samples = 2
value = [0, 2]
gini = 0.0
samples = 1
value = [1, 0]
gini = 0.0
samples = 1
value = [0, 1]

# K_Means

April 26, 2026

Problem Number 12:-

To create multiple clusters from column data entries of .csv file using k-means algorithm.

Name - Aditya Kumar Garg

University Roll No- 2023827

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

[2]:
```
data = pd.read_csv('driver-data.csv')
```

[3]:
```
from sklearn.cluster import KMeans
```

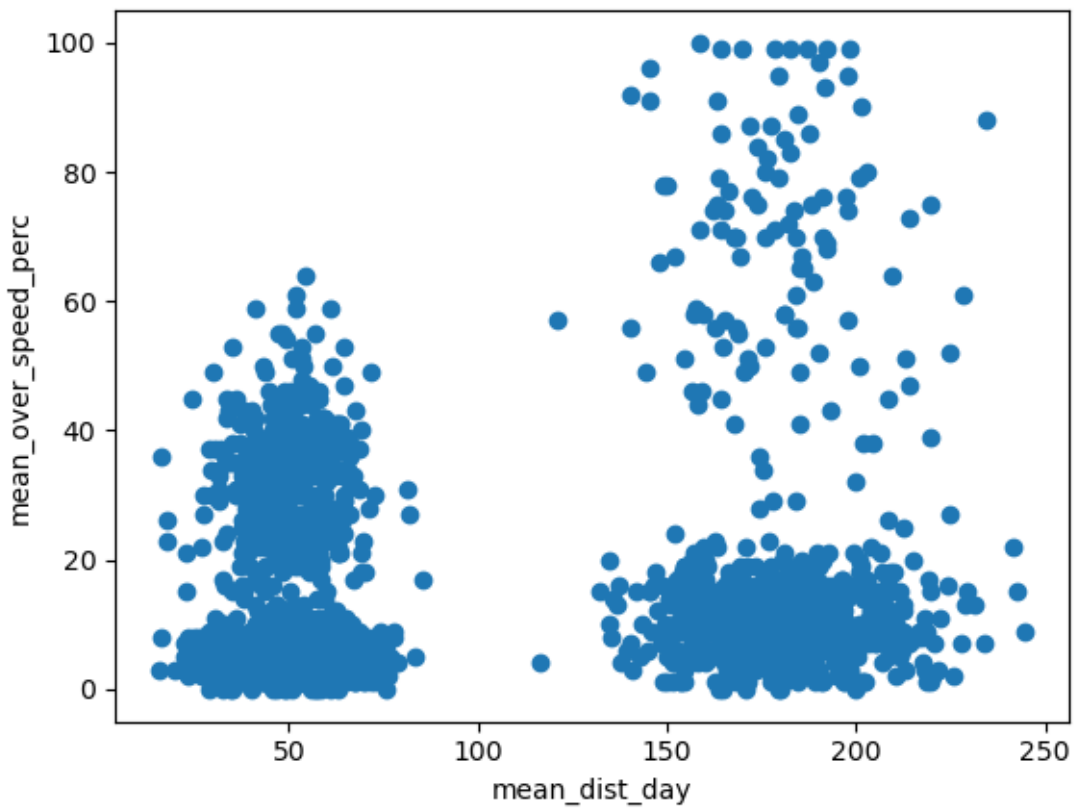
[4]:
```
data.head()
```

[4]:

| | id | mean_dist_day | mean_over_speed_perc |
|---|---|---|---|
| 0 | 3423311935 | 71.24 | 28 |
| 1 | 3423313212 | 52.53 | 25 |
| 2 | 3423313724 | 64.54 | 27 |
| 3 | 3423311373 | 55.69 | 22 |
| 4 | 3423310999 | 54.58 | 25 |

[5]:
```
X = data[['mean_dist_day','mean_over_speed_perc']]
```

[6]:
```
plt.scatter(X['mean_dist_day'],X['mean_over_speed_perc'])
#plt.scatter(X.iloc[:,0],X.iloc[:,1])
plt.xlabel('mean_dist_day')
plt.ylabel('mean_over_speed_perc')
plt.show()
```




[7]:
```
km = KMeans(n_clusters = 4)
```

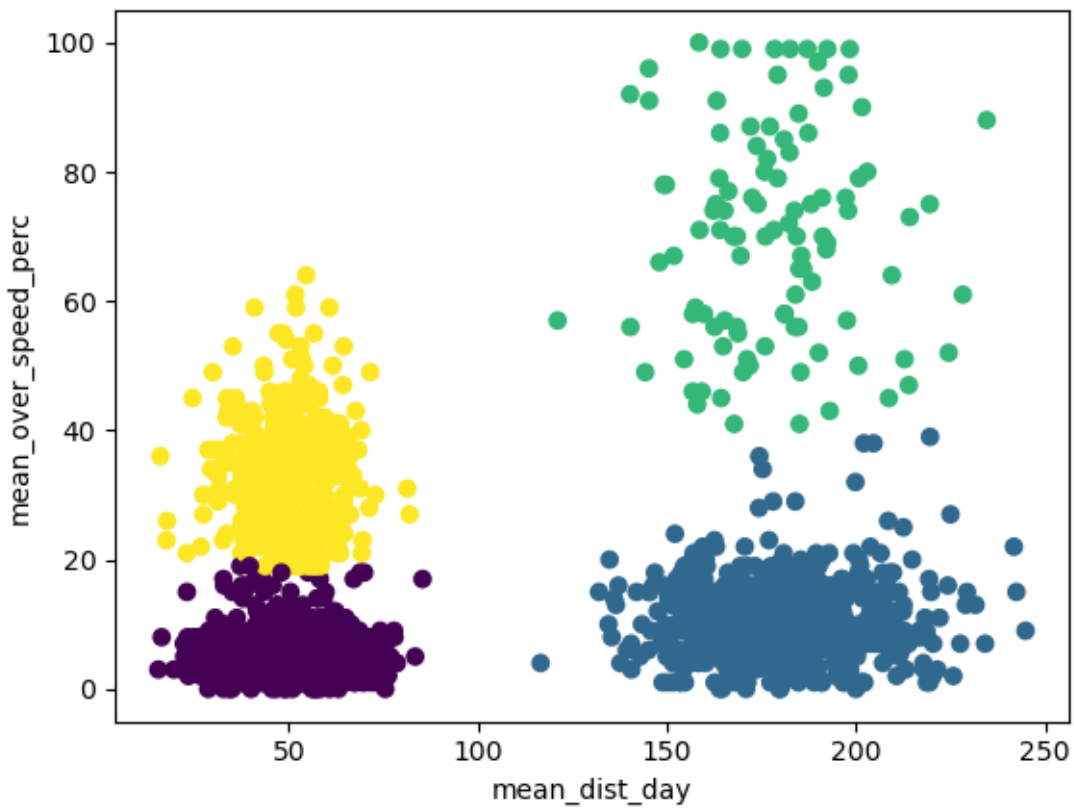
[8]:
```
km.fit(X)
```

[8]:
```
KMeans(n_clusters=4)
```

[9]:
```
print(km.labels_)
```

```
[3 3 3 … 1 1 1]
```

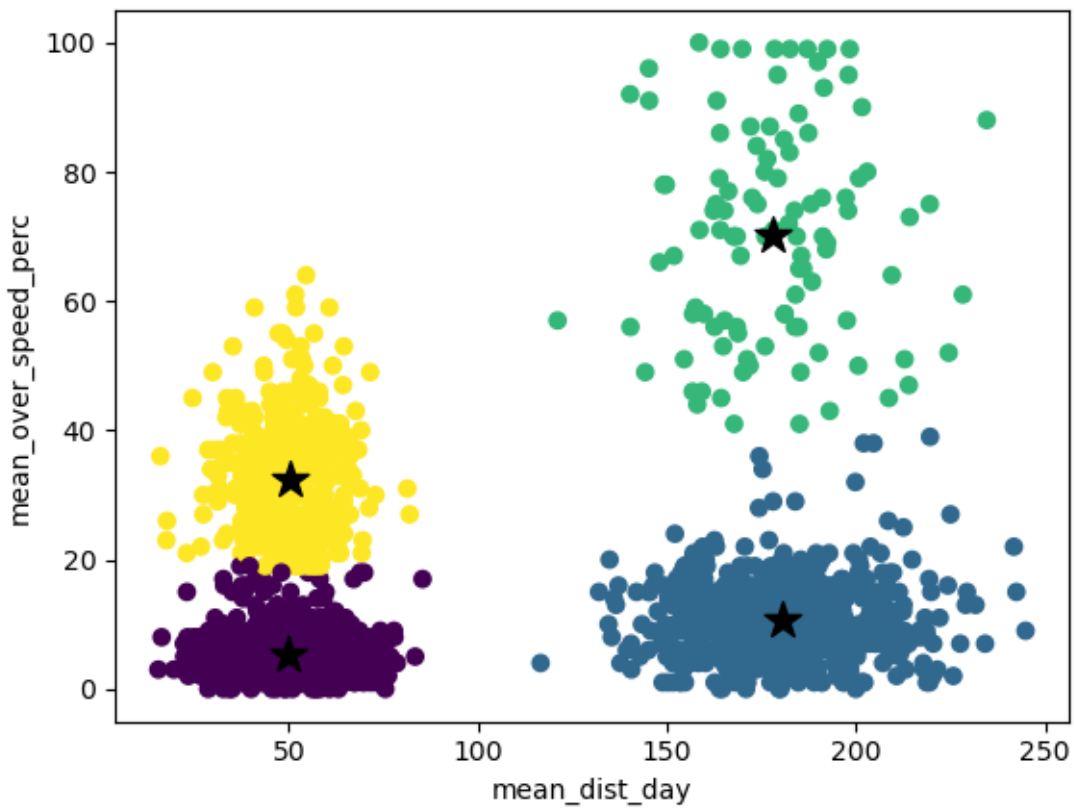
[10]:
```
plt.scatter(X['mean_dist_day'],X['mean_over_speed_perc'],c=km.labels_)
plt.xlabel('mean_dist_day')
plt.ylabel('mean_over_speed_perc')
plt.show()
```



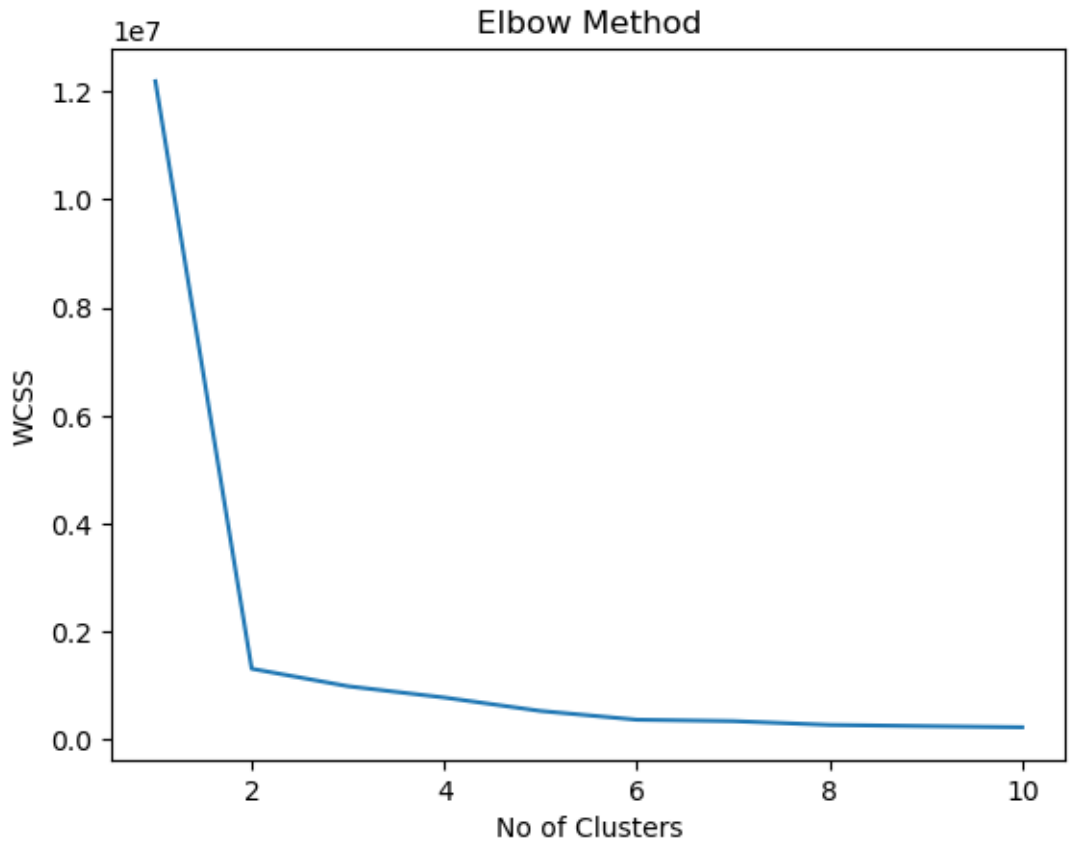

[11]:
```
centroids = km.cluster_centers_
print(centroids)
```

```
[[ 49.9799964    5.22434282]
 [180.34311782  10.52011494]
 [177.83509615  70.28846154]
 [ 50.49167849  32.49172577]]
```

[12]:
```
plt.scatter(X['mean_dist_day'],X['mean_over_speed_perc'],c=km.labels_)
plt.scatter(centroids[:,0],centroids[:
 ↪,1],marker='*',s=200,label='centroids',c='black')
plt.xlabel('mean_dist_day')
plt.ylabel('mean_over_speed_perc')
plt.show()
```




[13]:
```
# ELBOW METHOD
wcss=[]
for i in range(1,11):
    km=KMeans(n_clusters=i)
    km.fit(X)
    wcss.append(km.inertia_)
plt.plot(range(1,11),wcss)
plt.title("Elbow Method")
plt.xlabel("No of Clusters")
plt.ylabel("WCSS")
plt.plot()
plt.show()
```




[14]:
```
# SILHOUETTE METHOD
from sklearn.metrics import silhouette_score
km = KMeans(n_clusters=3)
labels = km.fit_predict(X)
score = silhouette_score(X,labels,metric='euclidean')
print(score)
```

```
0.44295862658622787
```

# Satistical_measures

April 26, 2026

Problem Number 13:-

To use healthcare dataset and form scatter plot with observed statistical measures.

Name - Aditya Kumar Garg

University Roll No- 2023827

[1]:
```
import pandas as pd
```

[2]:
```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
```

[3]:
```
df['target'] = data.target
print(df.head())
```

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  …  worst texture  worst perimeter  worst area  \
0                 0.07871  …          17.33           184.60      2019.0
1                 0.05667  …          23.41           158.80      1956.0
2                 0.05999  …          25.53           152.50      1709.0
3                 0.09744  …          26.50            98.87       567.7
4                 0.05883  …          16.67           152.20      1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
0            0.1622             0.6656           0.7119                0.2654
```

```
1            0.1238             0.1866           0.2416                 0.1860
2            0.1444             0.4245           0.4504                 0.2430
3            0.2098             0.8663           0.6869                 0.2575
4            0.1374             0.2050           0.4000                 0.1625

   worst symmetry  worst fractal dimension  target
0          0.4601                  0.11890       0
1          0.2750                  0.08902       0
2          0.3613                  0.08758       0
3          0.6638                  0.17300       0
4          0.2364                  0.07678       0

[5 rows x 31 columns]
```

[4]:
```python
print("Mean:\n", df.mean())
print("\nMedian:\n", df.median())
print("\nStandard Deviation:\n", df.std())
```

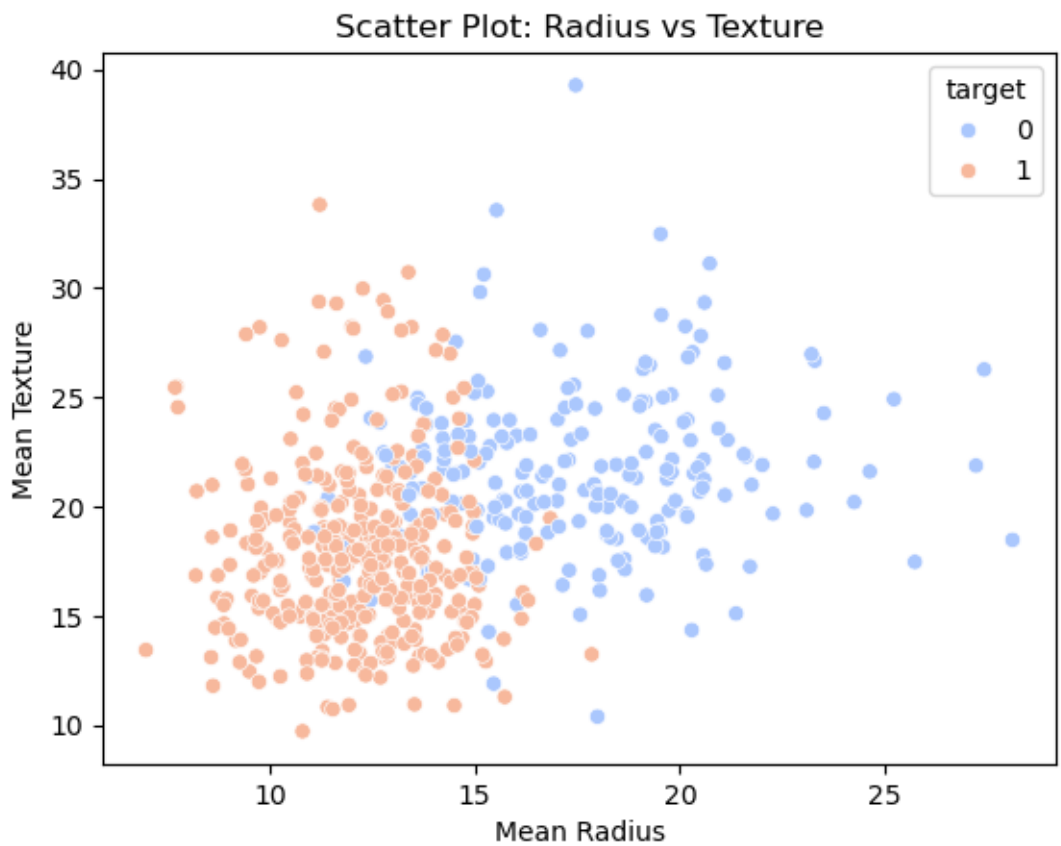
```
Mean:
 mean radius                 14.127292
mean texture                19.289649
mean perimeter              91.969033
mean area                  654.889104
mean smoothness              0.096360
mean compactness             0.104341
mean concavity               0.088799
mean concave points          0.048919
mean symmetry                0.181162
mean fractal dimension       0.062798
radius error                 0.405172
texture error                1.216853
perimeter error              2.866059
area error                  40.337079
smoothness error             0.007041
compactness error            0.025478
concavity error              0.031894
concave points error         0.011796
symmetry error               0.020542
fractal dimension error      0.003795
worst radius                16.269190
worst texture               25.677223
worst perimeter            107.261213
worst area                 880.583128
worst smoothness             0.132369
worst compactness            0.254265
worst concavity              0.272188
worst concave points         0.114606
worst symmetry               0.290076
```

```
worst fractal dimension      0.083946
target                       0.627417
dtype: float64

Median:
 mean radius                 13.370000
mean texture                18.840000
mean perimeter              86.240000
mean area                  551.100000
mean smoothness              0.095870
mean compactness             0.092630
mean concavity               0.061540
mean concave points          0.033500
mean symmetry                0.179200
mean fractal dimension       0.061540
radius error                 0.324200
texture error                1.108000
perimeter error              2.287000
area error                  24.530000
smoothness error             0.006380
compactness error            0.020450
concavity error              0.025890
concave points error         0.010930
symmetry error               0.018730
fractal dimension error      0.003187
worst radius                14.970000
worst texture               25.410000
worst perimeter             97.660000
worst area                 686.500000
worst smoothness             0.131300
worst compactness            0.211900
worst concavity              0.226700
worst concave points         0.099930
worst symmetry               0.282200
worst fractal dimension      0.080040
target                       1.000000
dtype: float64

Standard Deviation:
 mean radius                  3.524049
mean texture                 4.301036
mean perimeter              24.298981
mean area                  351.914129
mean smoothness              0.014064
mean compactness             0.052813
mean concavity               0.079720
mean concave points          0.038803
mean symmetry                0.027414
```

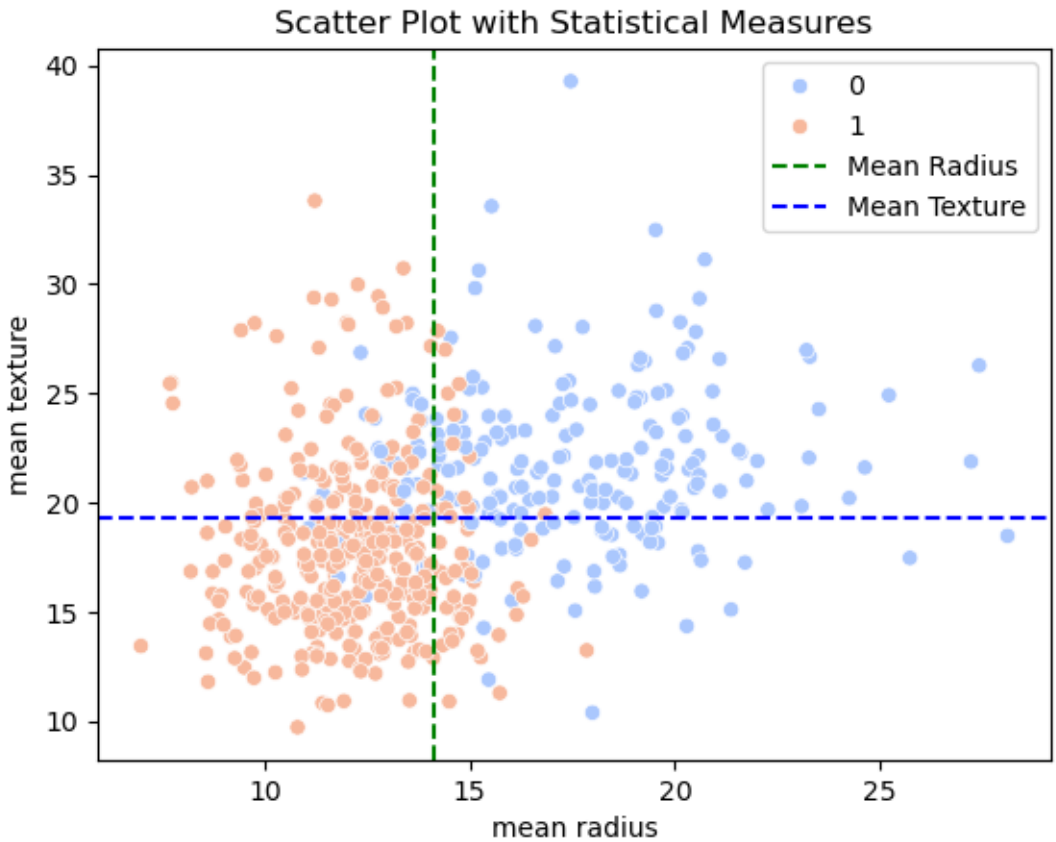
```
mean fractal dimension       0.007060
radius error                 0.277313
texture error                0.551648
perimeter error              2.021855
area error                  45.491006
smoothness error             0.003003
compactness error            0.017908
concavity error              0.030186
concave points error         0.006170
symmetry error               0.008266
fractal dimension error      0.002646
worst radius                 4.833242
worst texture                6.146258
worst perimeter             33.602542
worst area                 569.356993
worst smoothness             0.022832
worst compactness            0.157336
worst concavity              0.208624
worst concave points         0.065732
worst symmetry               0.061867
worst fractal dimension      0.018061
target                       0.483918
dtype: float64
```

[5]:
```
import seaborn as sns
import matplotlib.pyplot as plt
```

[6]:
```
sns.scatterplot(x=df['mean radius'], y=df['mean texture'], hue=df['target'],␣
 ↪palette='coolwarm')
plt.title("Scatter Plot: Radius vs Texture")
plt.xlabel("Mean Radius")
plt.ylabel("Mean Texture")
plt.show()
```




[8]:
```
mean_x = df['mean radius'].mean()
mean_y = df['mean texture'].mean()
sns.scatterplot(x=df['mean radius'],y=df['mean␣
↪texture'],hue=df['target'],palette='coolwarm')
plt.axvline(mean_x, color='green', linestyle='--', label='Mean Radius')
plt.axhline(mean_y, color='blue', linestyle='--', label='Mean Texture')
plt.legend()
plt.title("Scatter Plot with Statistical Measures")
plt.show()
```


Scatter Plot with Statistical Measures
0
1
Mean Radius
Mean Texture
mean texture
mean radius
40
35
30
25
20
15
10
10
15
20
25

# problem_14

April 26, 2026

Problem Number 14:-

Implement any classical ML techniques to a real-world problem and perform following operations:

1) Choose a dataset
2) Preprocess
3) Train models
4) Analyze results

Name - Aditya Kumar Garg

University Roll No - 2023827

[1]:
```
import pandas as pd
```

[2]:
```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
print(df.head())
```

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  …  worst texture  worst perimeter  worst area  \
0                 0.07871  …          17.33           184.60      2019.0
1                 0.05667  …          23.41           158.80      1956.0
2                 0.05999  …          25.53           152.50      1709.0
3                 0.09744  …          26.50            98.87       567.7
4                 0.05883  …          16.67           152.20      1575.0
```

```
   worst smoothness  worst compactness  worst concavity  worst concave points  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   worst symmetry  worst fractal dimension  target
0          0.4601                  0.11890       0
1          0.2750                  0.08902       0
2          0.3613                  0.08758       0
3          0.6638                  0.17300       0
4          0.2364                  0.07678       0

[5 rows x 31 columns]
```

[3]:
```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X = df.drop('target', axis=1)
y = df['target']
# Train-test split (important)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

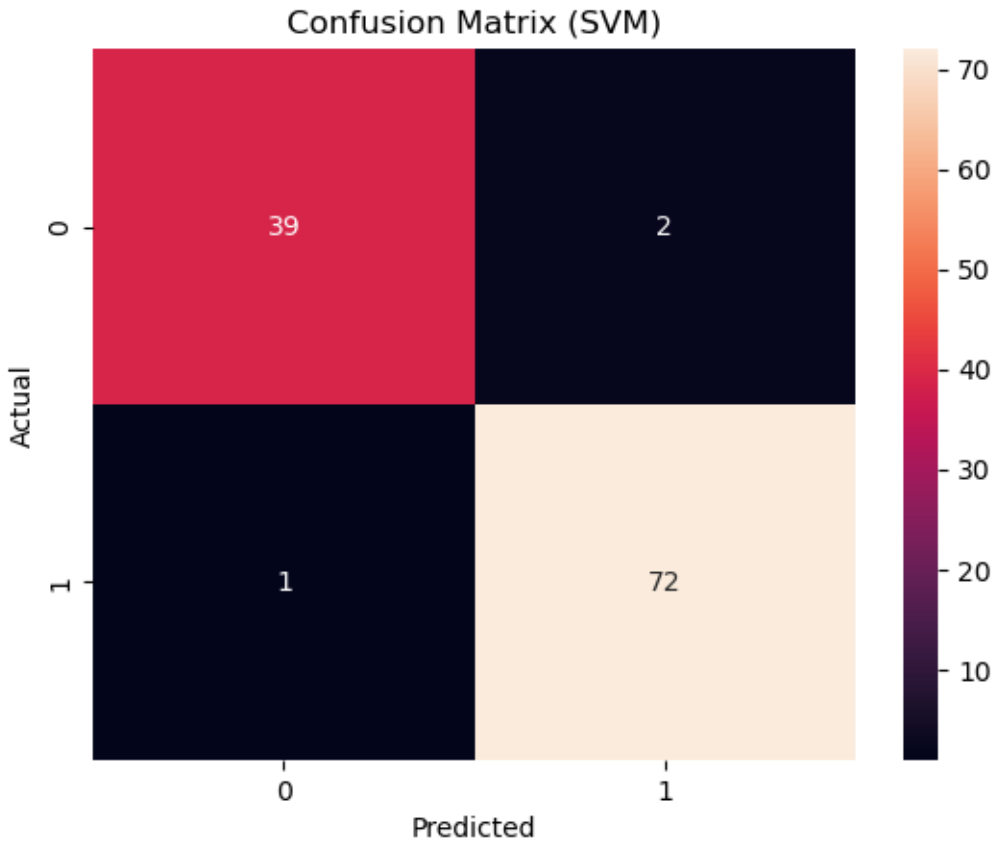
[4]:
```python
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
# Logistic Regression
lr_model = LogisticRegression()
lr_model.fit(X_train, y_train)
y_pred_lr = lr_model.predict(X_test)
# SVM (RBF kernel)
svm_model = SVC(kernel='rbf')
svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)
```

[5]:
```python
from sklearn.metrics import accuracy_score, confusion_matrix,␣
↪classification_report
def evaluate(name, y_test, y_pred):
    print(f"\n{name}")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
evaluate("Logistic Regression", y_test, y_pred_lr)
evaluate("SVM (RBF)", y_test, y_pred_svm)
```

```
Logistic Regression
Accuracy: 0.9649122807017544
Confusion Matrix:
 [[37  4]
 [ 0 73]]
Classification Report:
               precision    recall  f1-score   support

           0       1.00      0.90      0.95        41
           1       0.95      1.00      0.97        73

    accuracy                           0.96       114
   macro avg       0.97      0.95      0.96       114
weighted avg       0.97      0.96      0.96       114


SVM (RBF)
Accuracy: 0.9736842105263158
Confusion Matrix:
 [[39  2]
 [ 1 72]]
Classification Report:
               precision    recall  f1-score   support

           0       0.97      0.95      0.96        41
           1       0.97      0.99      0.98        73

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114
```

[6]:
```
import seaborn as sns
import matplotlib.pyplot as plt
cm = confusion_matrix(y_test, y_pred_svm)
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix (SVM)")
plt.show()
```


Confusion Matrix (SVM)
39
2
1
72
0
1
Actual
0
1
Predicted
70
60
50
40
30
20
10